Data Structure & Algorithms

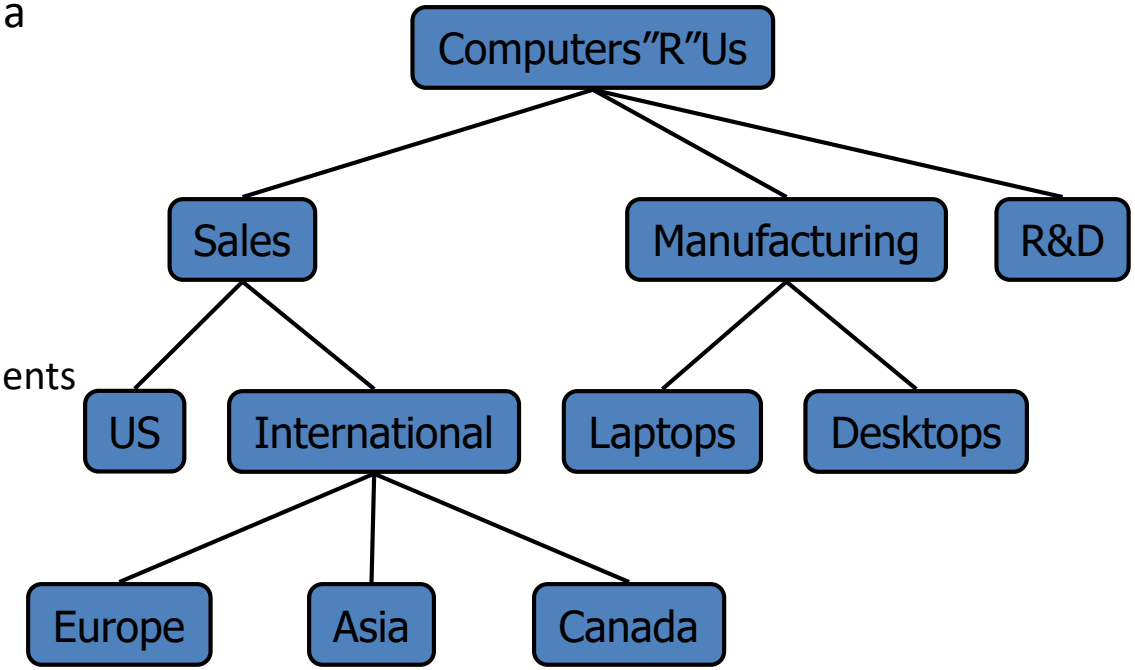
Submitted By:-

Maanvika
Assistant Prof - CSE

Unit 4
Tree and Its Types

What is a Tree

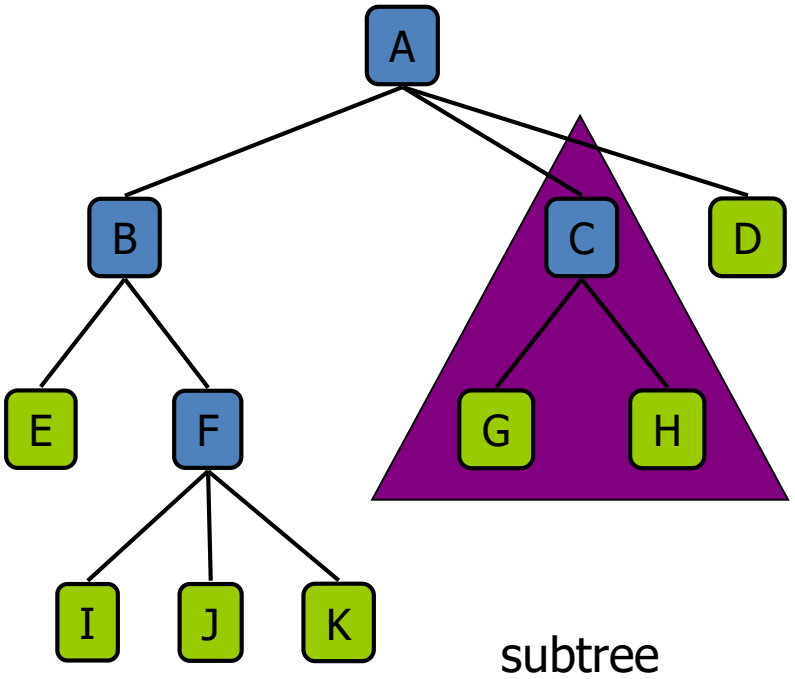
- A tree is a finite nonempty set of elements.
- It is an abstract model of a hierarchical structure.
- consists of nodes with a parent-child relation.
- Applications:
  - Organization charts
  - File systems
  - Programming environments



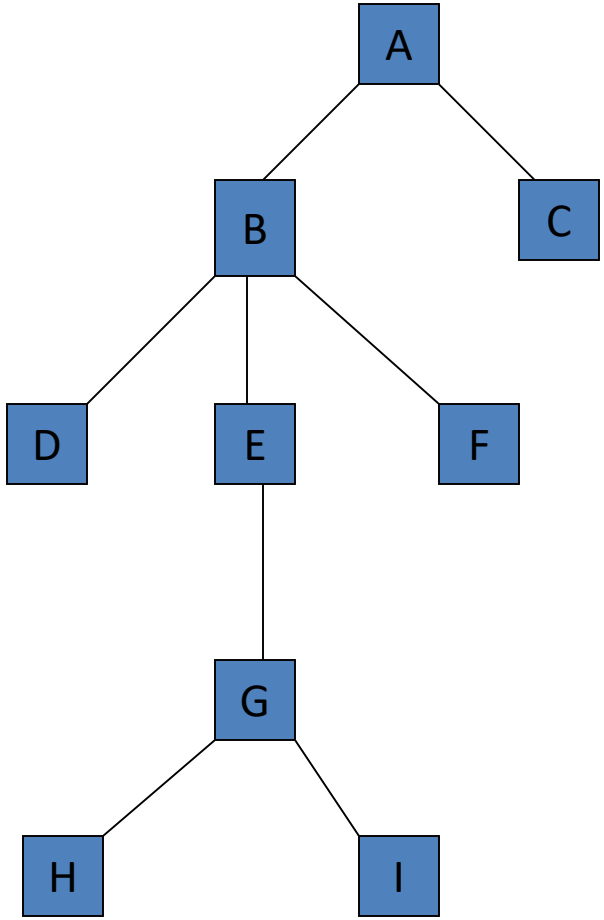
Tree Terminology

- **Root**: node without parent (A)
- **Siblings**: nodes share the same parent
- **Internal node**: node with at least one child (A, B, C, F)
- **External node** (leaf): node without children (E, I, J, K, G, H, D)
- **Ancestors** of a node: parent, grandparent, grand-grandparent, etc.
- **Descendant** of a node: child, grandchild, grand-grandchild, etc.
- **Depth** of a node: number of ancestors
- **Height** of a tree: maximum depth of any node (3)
- **Degree** of a node: the number of its children
- **Degree** of a tree: the maximum number of its node.

- **Subtree**: tree consisting of a node and its descendants



Tree Properties



Property	Value
Number of nodes	
Height	
Root Node	
Leaves	
Interior nodes	
Ancestors of H	
Descendants of B	
Siblings of E	
Right subtree of A	
Degree of this tree	

Tree ADT

- We use positions to abstract nodes
- Generic methods:
  - integer **size**()
  - boolean **isEmpty**()
  - objectIterator **elements**()
  - positionIterator **positions**()
- Accessor methods:
  - position **root**()
  - position **parent**(p)
  - positionIterator **children**(p)

- Query methods:
  - boolean **isInternal**(p)
  - boolean **isExternal**(p)
  - boolean **isRoot**(p)
- Update methods:
  - **swapElements**(p, q)
  - object **replaceElement**(p, o)
- Additional update methods may be defined by data structures implementing the Tree ADT

Intuitive Representation of Tree Node

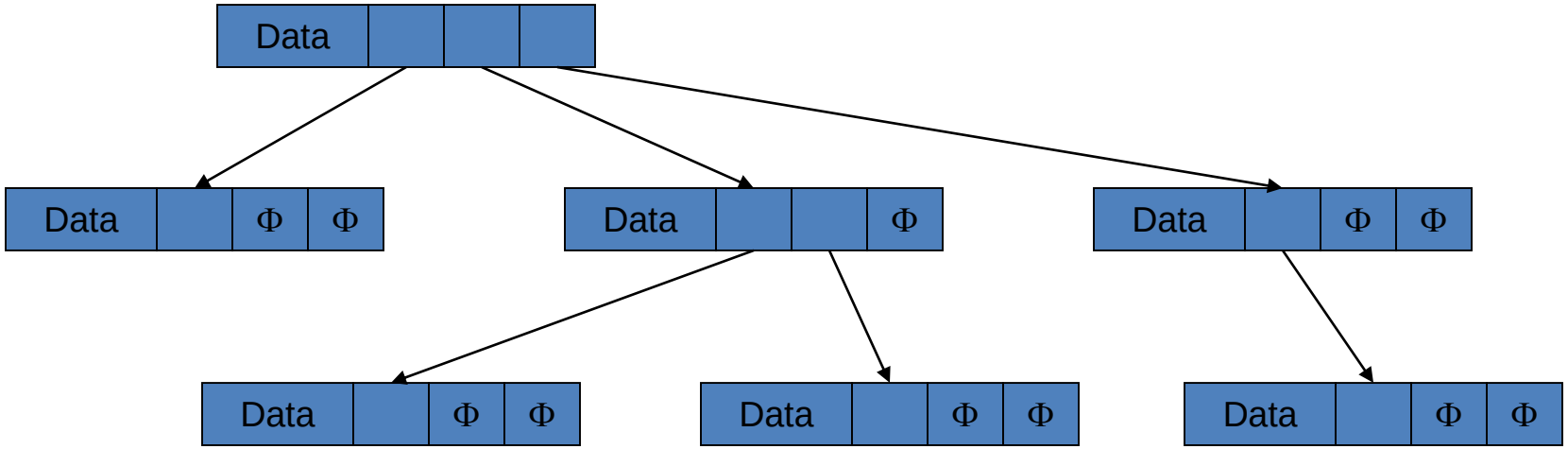
- List Representation
  - (A (B (E (K, L), F), C (G), D (H (M), I, J)))
  - The root comes first, followed by a list of links to sub-trees

How many link fields are needed in such a representation?

Data	Link 1	Link 2	…	Link n

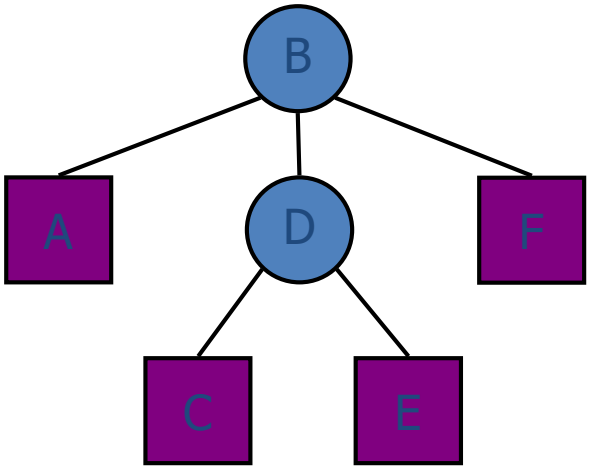
Trees

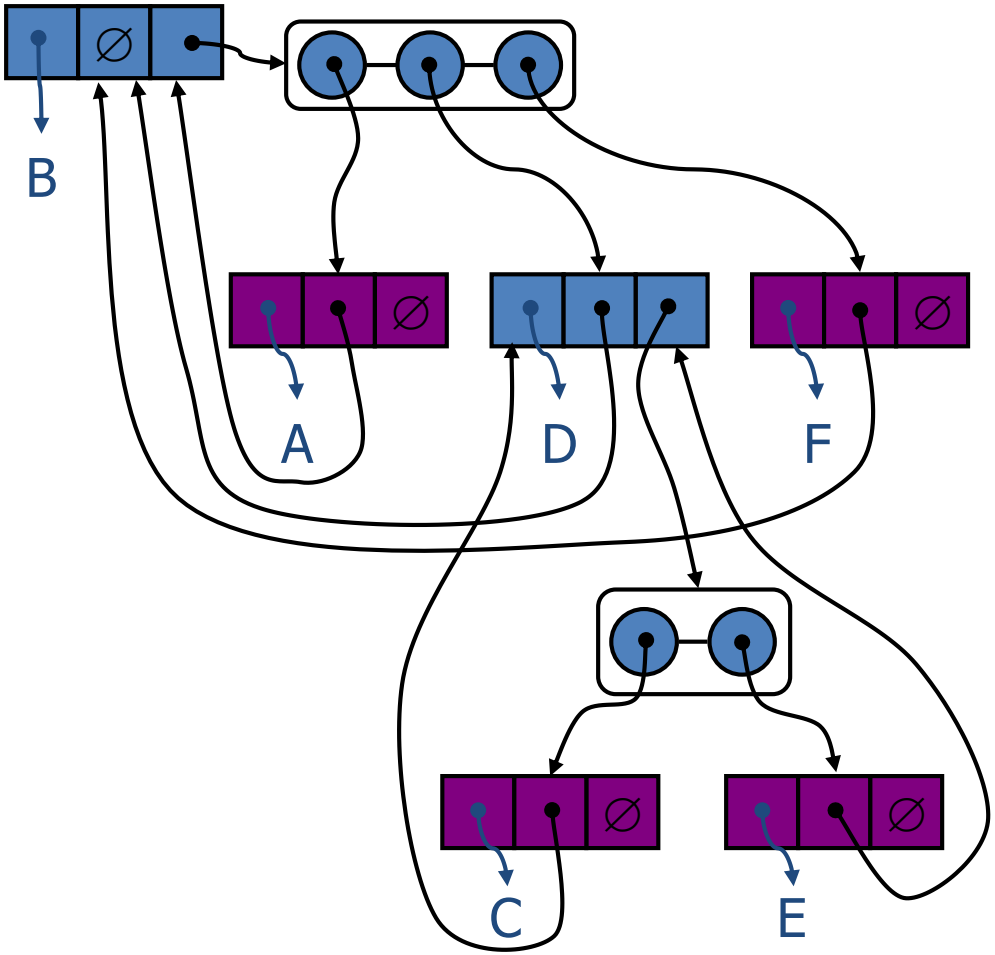
- Every tree node:
  - object – useful information
  - children – pointers to its children



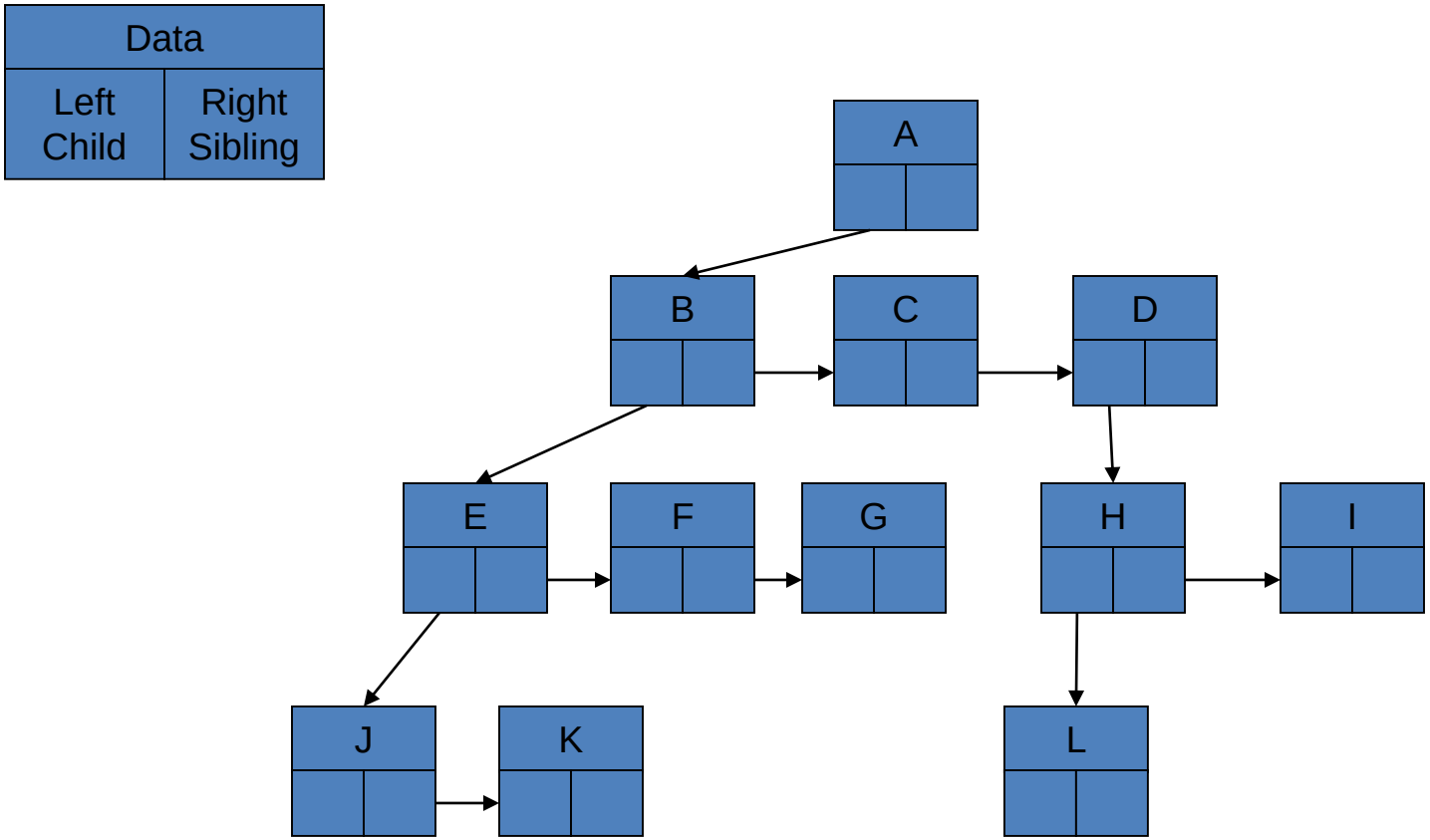
A Tree Representation

- A node is represented by an object storing
  - Element
  - Parent node
  - Sequence of children nodes





Left Child, Right Sibling Representation



Tree Traversal

- Two main methods:
  - Preorder
  - Postorder
- Recursive definition

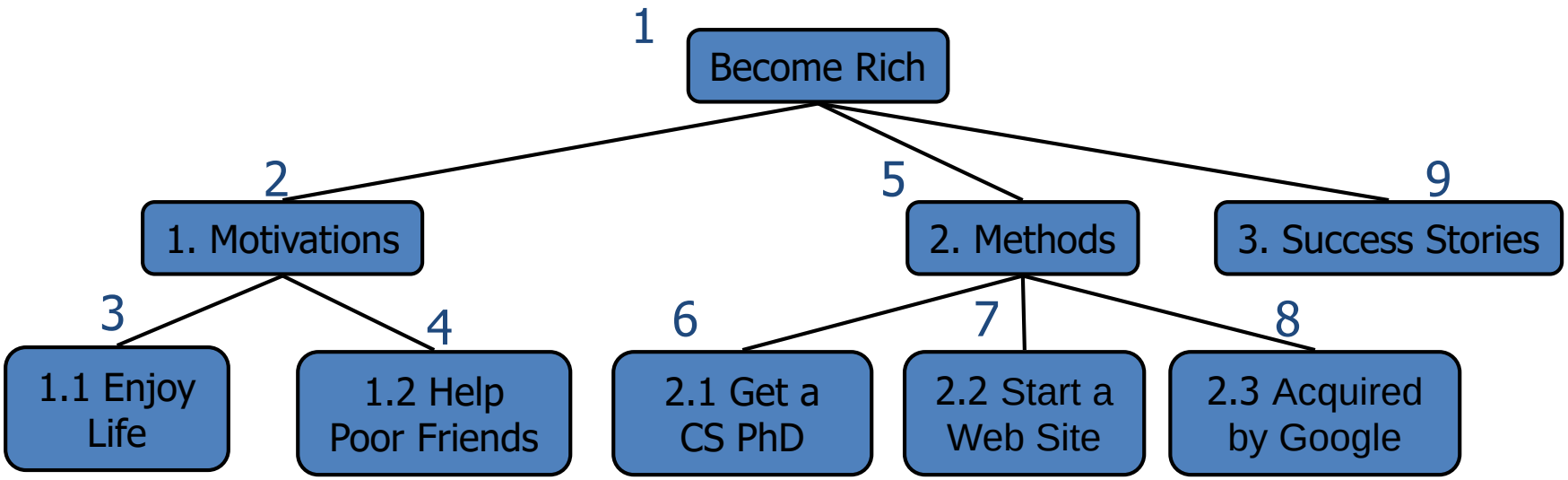
- Preorder:
  - visit the root
  - traverse in preorder the children (subtrees)

- Postorder
  - traverse in postorder the children (subtrees)
  - visit the root

Preorder Traversal

- A traversal visits the nodes of a tree in a systematic manner
- In a preorder traversal, a node is visited before its descendants
- Application: print a structured document

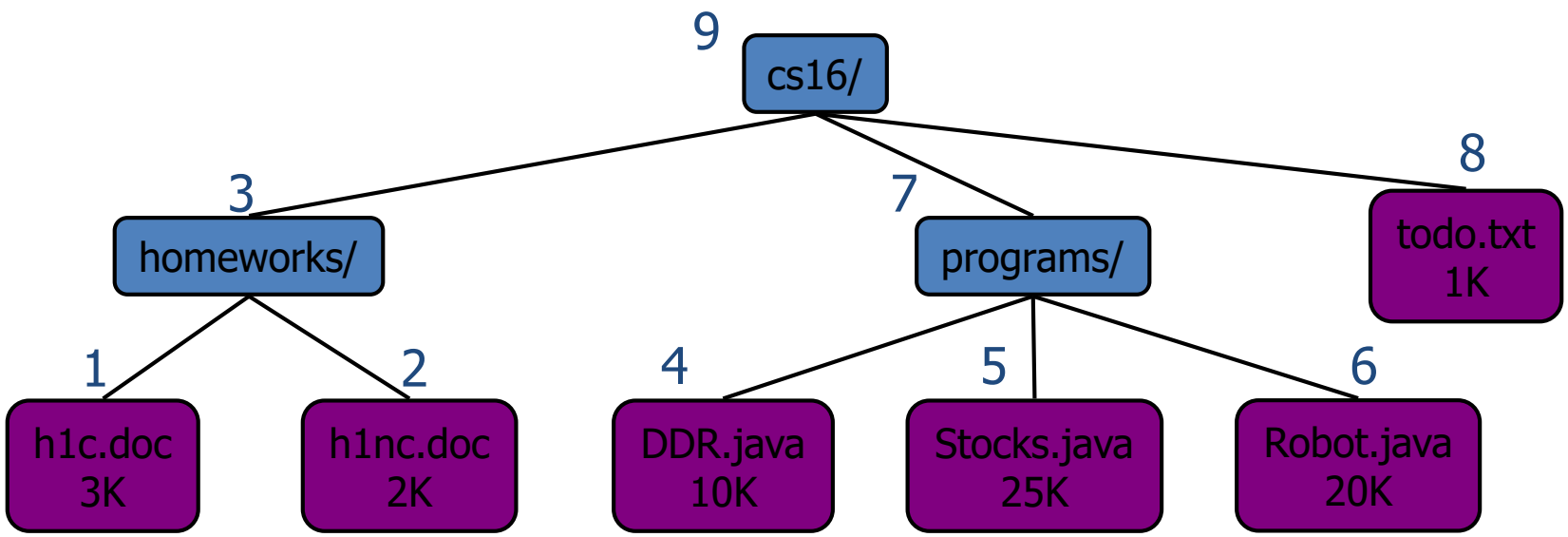
Algorithm ***preOrder***(***v***)
    visit(***v***)
    for each child ***w*** of ***v***
        preorder (***w***)



Postorder Traversal

- In a postorder traversal, a node is visited after its descendants
- Application: compute space used by files in a directory and its subdirectories

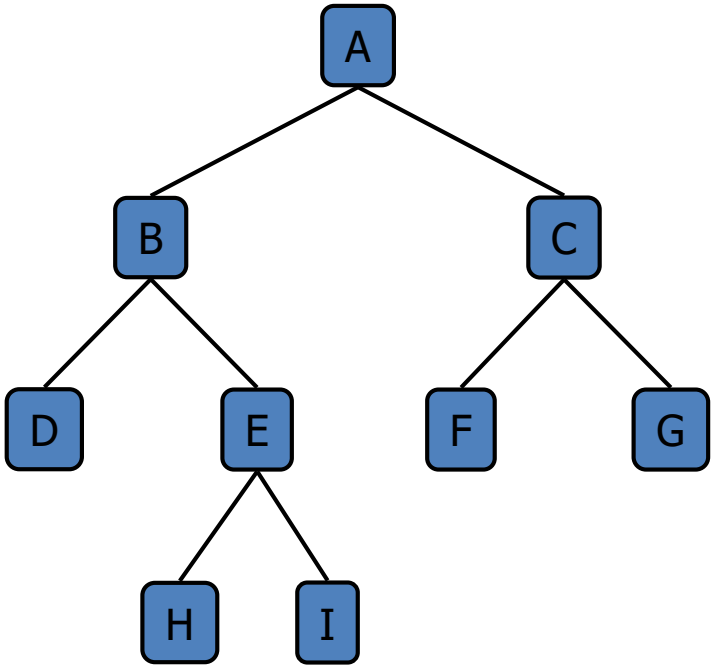
Algorithm ***postOrder***(***v***)
  for each child ***w*** of ***v***
    postOrder (***w***)
  visit(***v***)



Binary Tree

- A binary tree is a tree with the following properties:
  - Each internal node has at most two children (degree of two)
  - The children of a node are an ordered pair
- We call the children of an internal node left child and right child
- Alternative recursive definition: a binary tree is either
  - a tree consisting of a single node, OR
  - a tree whose root has an ordered pair of children, each of which is a binary tree

- Applications:
  - arithmetic expressions
  - decision processes
  - searching

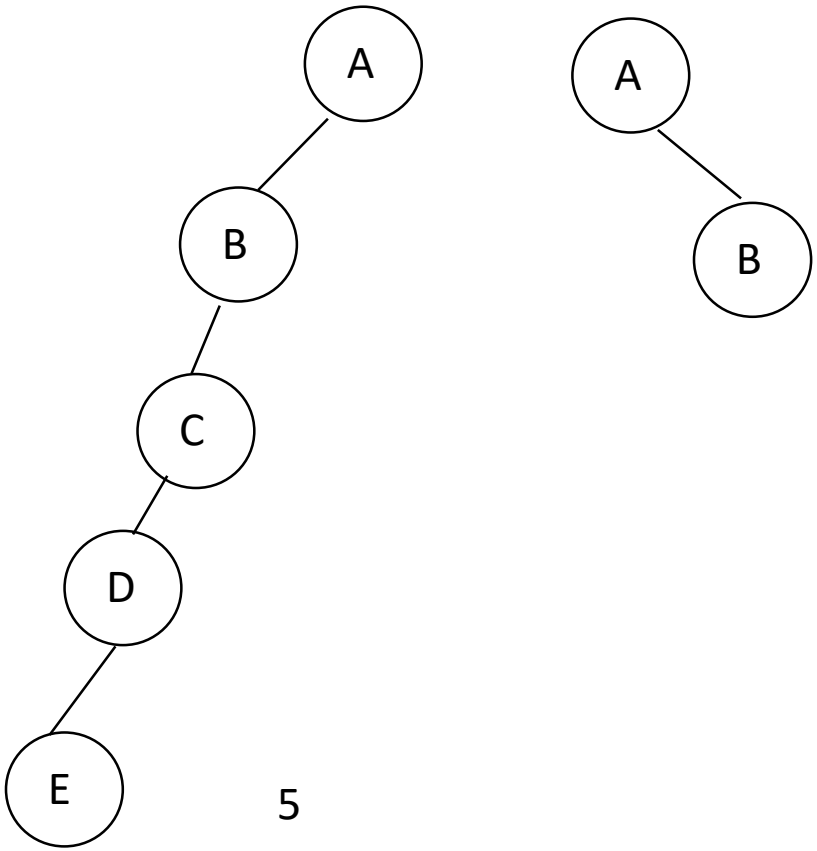


BinaryTree ADT

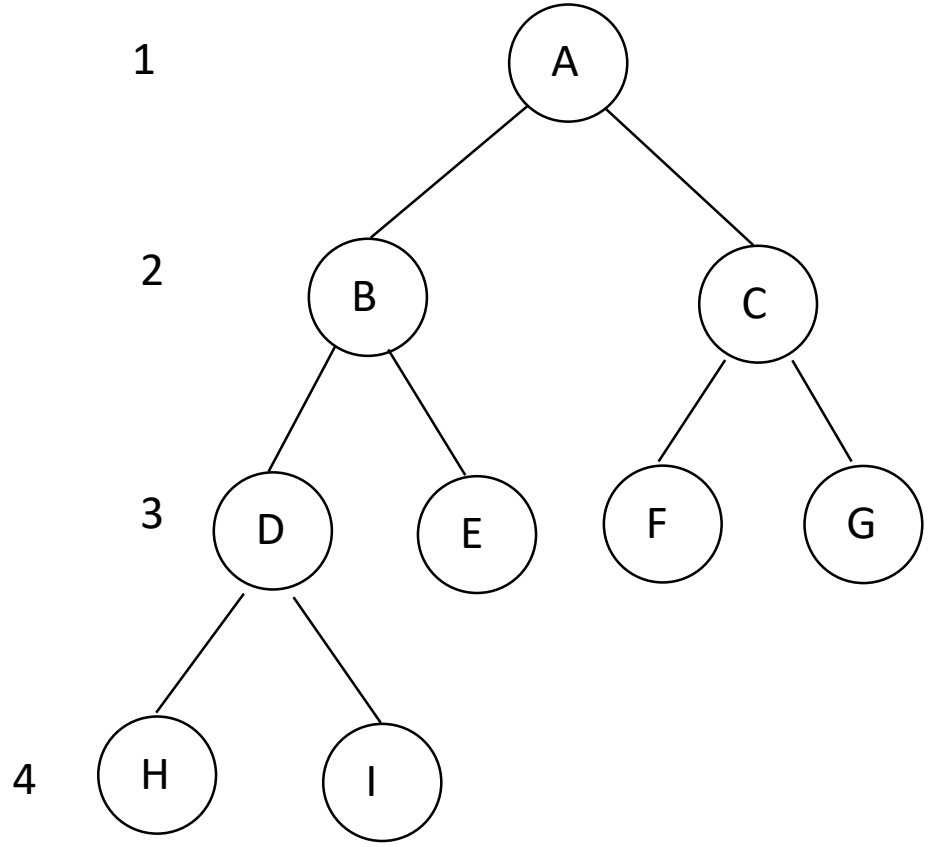
- The BinaryTree ADT extends the Tree ADT, i.e., it inherits all the methods of the Tree ADT
- Additional methods:
  - position leftChild(p)
  - position rightChild(p)
  - position sibling(p)
- Update methods may be defined by data structures implementing the BinaryTree ADT

Examples of the Binary Tree

Skewed Binary Tree



Complete Binary Tree



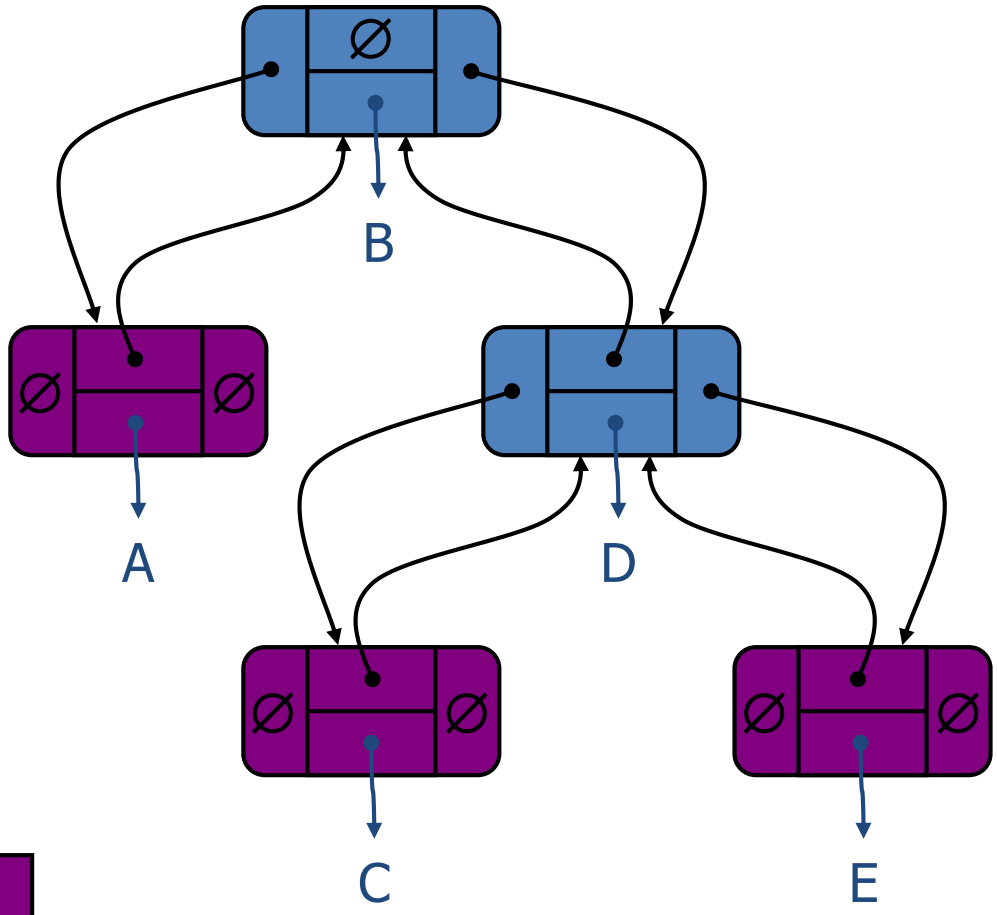
Differences Between A Tree and A Binary Tree

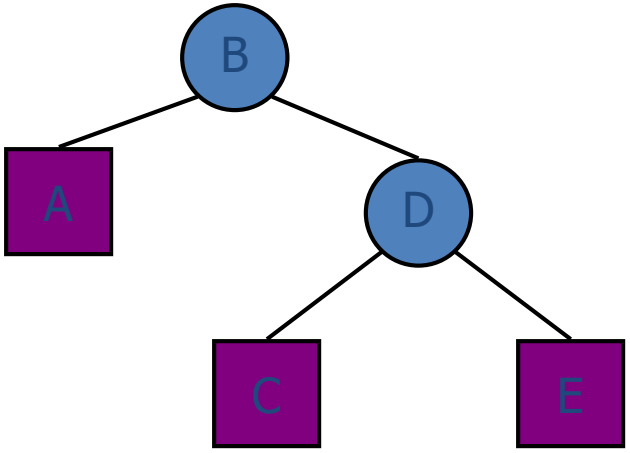
- The subtrees of a binary tree are ordered; those of a tree are not ordered.

- Are different when viewed as binary trees.
- Are the same when viewed as trees.

Data Structure for Binary Trees

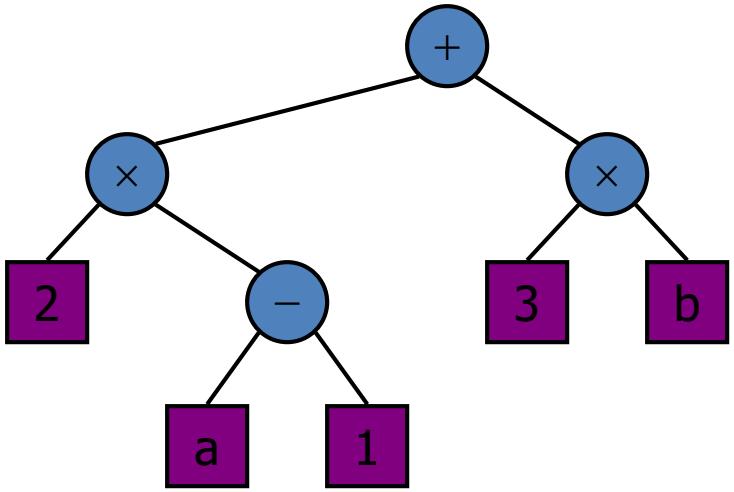
- A node is represented by an object storing
  - Element
  - Parent node
  - Left child node
  - Right child node





Arithmetic Expression Tree

- Binary tree associated with an arithmetic expression
  - internal nodes: operators
  - external nodes: operands
- Example: arithmetic expression tree for the expression $(2 \times (a - 1) + (3 \times b))$



Maximum Number of Nodes in a Binary Tree

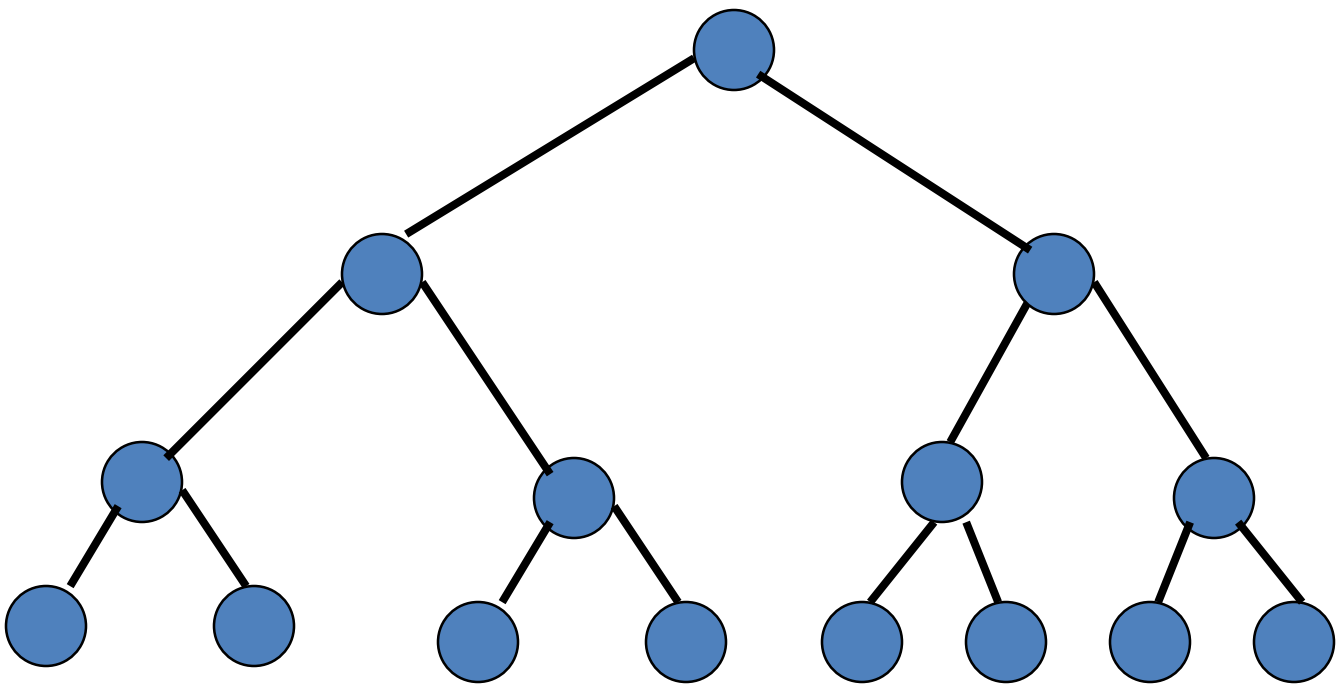
- The maximum number of nodes on depth i of a binary tree is 2^i, i>=0.
- The maximum nubmer of nodes in a binary tree of height k is $2^{k+1}-1$, k>=0.

Prove by induction.

$$\sum_{i=0}^{k} 2^i = 2^{k+1} - 1$$

Full Binary Tree

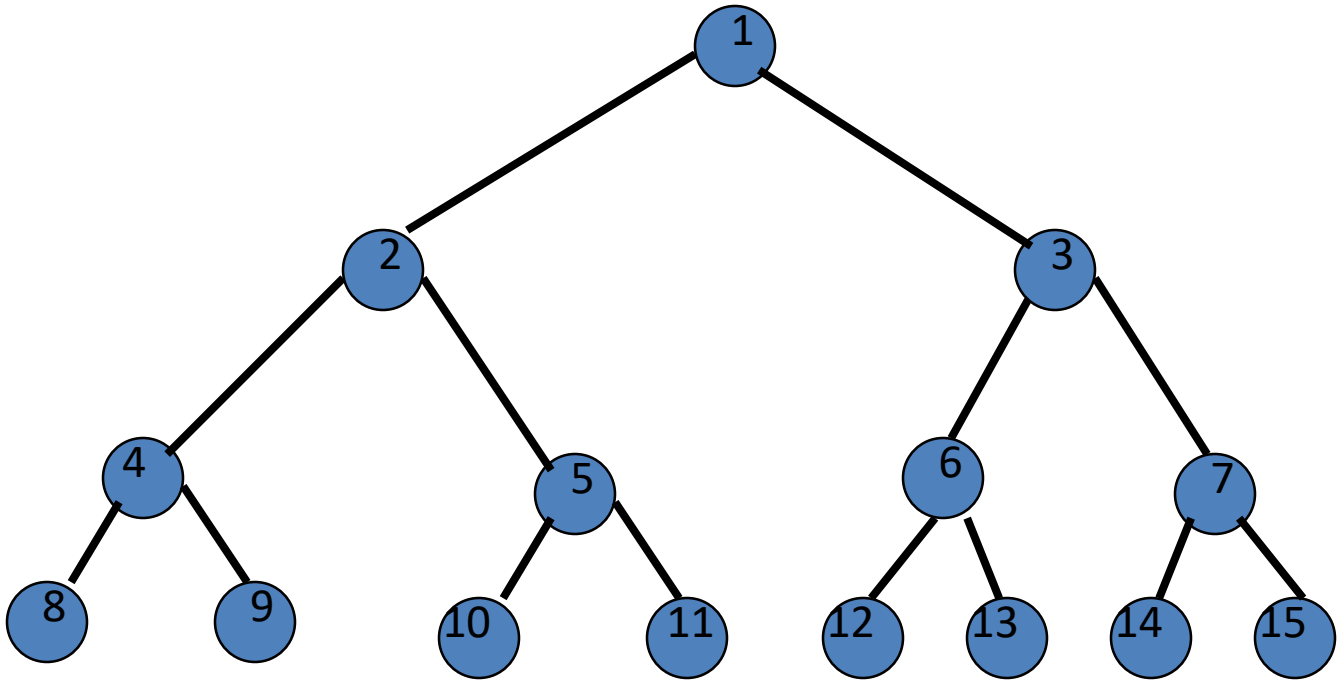
- A full binary tree of a given height k has $2^{k+1}-1$ nodes.



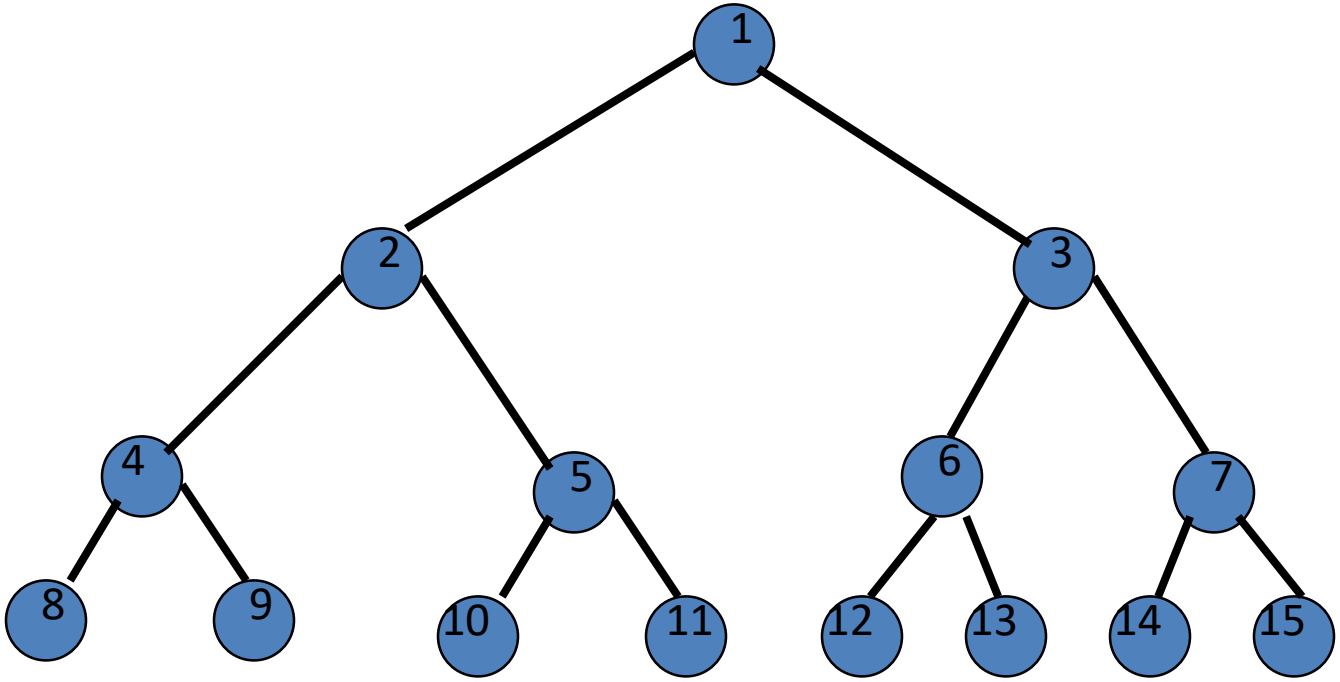
Height 3 full binary tree.

Labeling Nodes In A Full Binary Tree

- Label the nodes 1 through $2^{k+1} - 1$.
- Label by levels from top to bottom.
- Within a level, label from left to right.

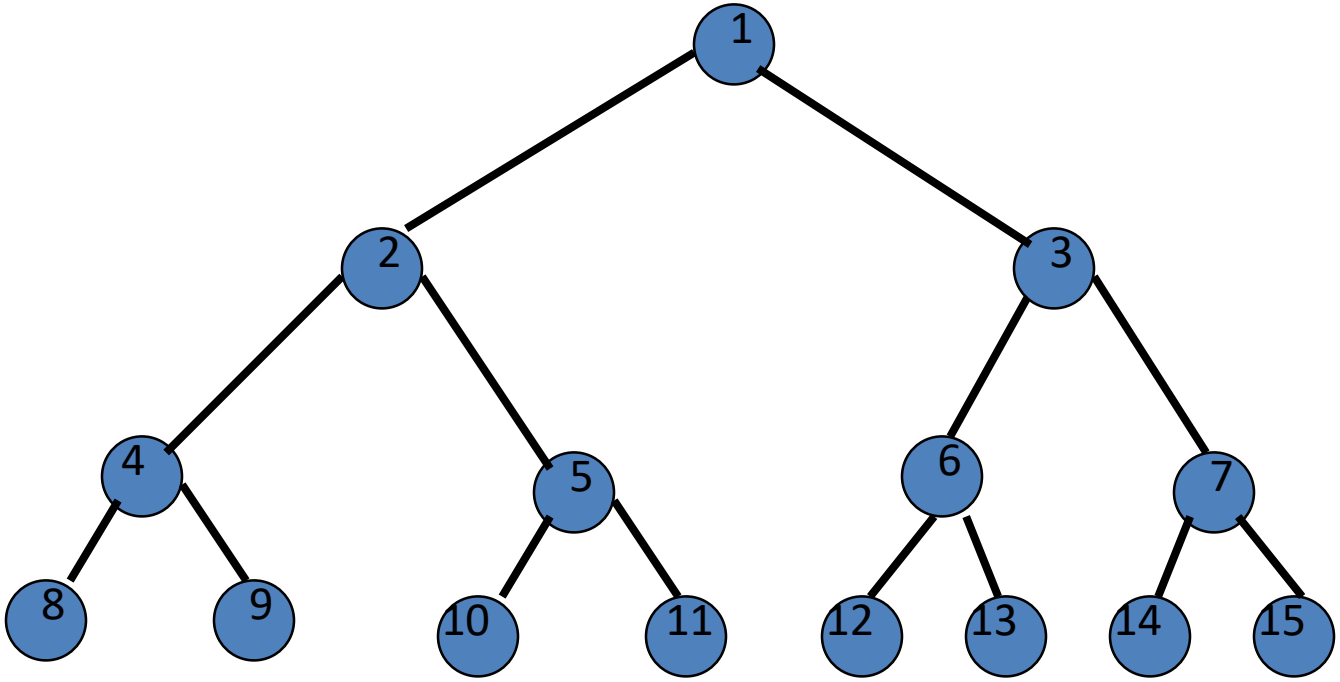


Node Number Properties



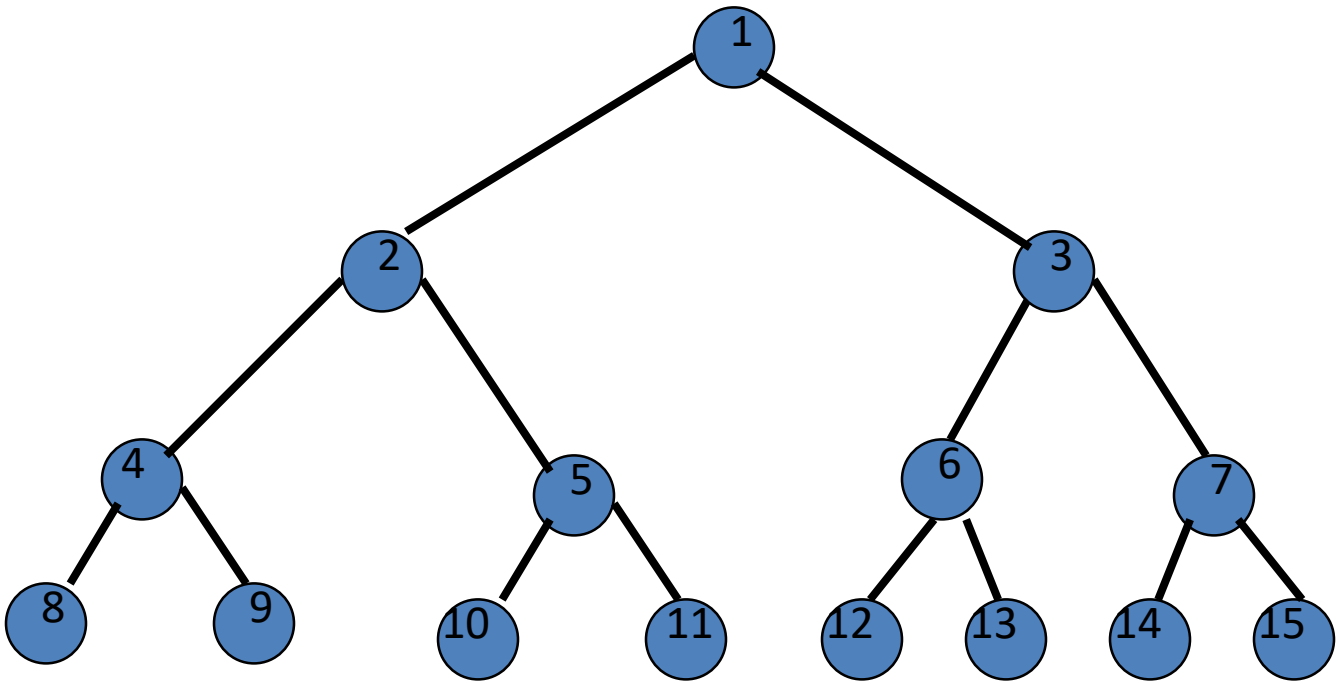
- Parent of node i is node i / 2, unless i = 1.
- Node 1 is the root and has no parent.

Node Number Properties



- Left child of node i is node $2i$, unless $2i > n$, where n is the number of nodes.
- If $2i > n$, node i has no left child.

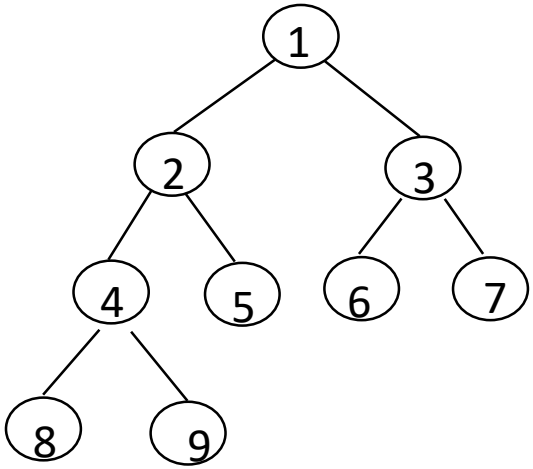
Node Number Properties



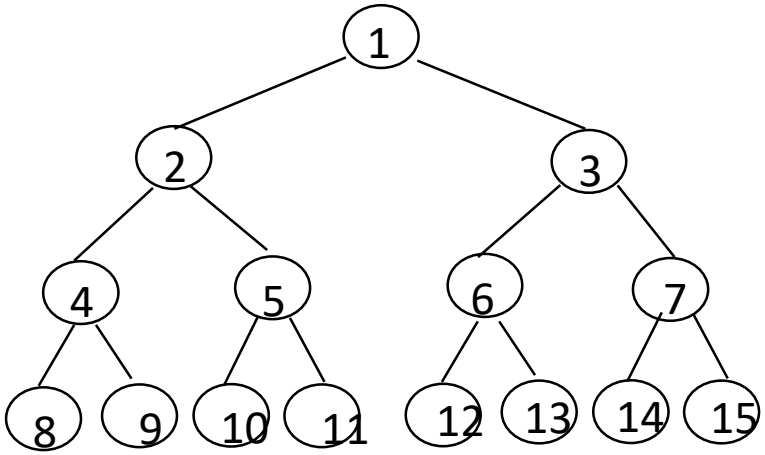
- Right child of node i is node $2i+1$, unless $2i+1 > n$, where n is the number of nodes.
- If $2i+1 > n$, node i has no right child.

Complete Binary Trees

- A labeled binary tree containing the labels 1 to n with root 1, branches leading to nodes labeled 2 and 3, branches from these leading to 4, 5 and 6, 7, respectively, and so on.
- A binary tree with *n* nodes and level *k* is complete *iff* its nodes correspond to the nodes numbered from 1 to *n* in the full binary tree of level *k*.



Complete binary tree



Full binary tree of depth 3

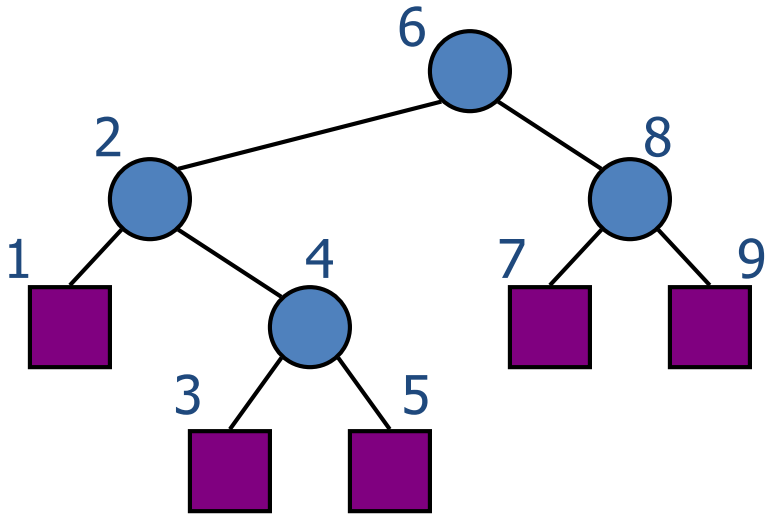
Binary Tree Traversals

- Let l, R, and r stand for moving left, visiting the node, and moving right.
- There are six possible combinations of traversal
  - lRr, lrR, Rlr, Rrl, rRl, rlR
- Adopt convention that we traverse left before right, only 3 traversals remain
  - lRr, lrR, Rlr
  - inorder, postorder, preorder

Inorder Traversal

- In an inorder traversal a node is visited after its left subtree and before its right subtree

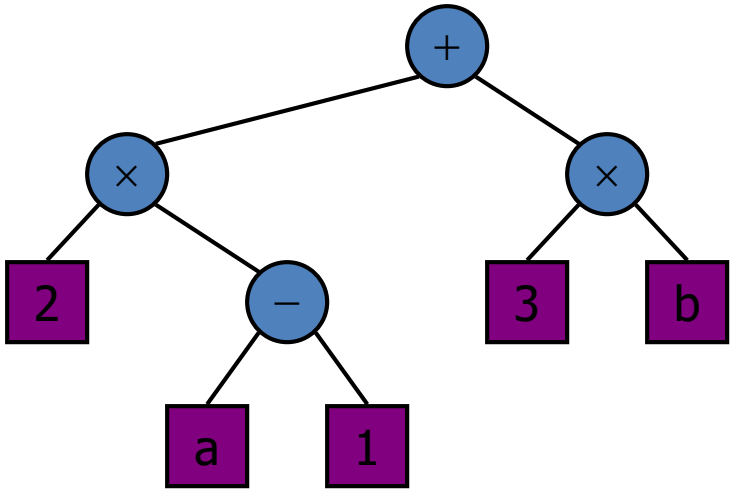
```
Algorithm inOrder(v)
    if isInternal (v)
        inOrder (leftChild (v))
    visit(v)
    if isInternal (v)
        inOrder (rightChild (v))
```



Print Arithmetic Expressions

- Specialization of an inorder traversal
  - print operand or operator when visiting node
  - print "(" before traversing left subtree
  - print ")" after traversing right subtree

```
Algorithm inOrder (v)
    if isInternal (v){
        print("(")
        inOrder (leftChild (v))}
    print(v.element ())
    if isInternal (v){
        inOrder (rightChild (v))
        print (")")}
```

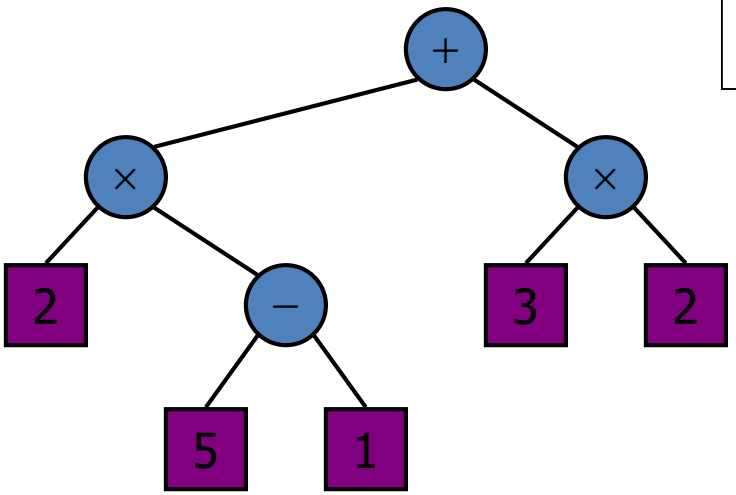


$$((2 \times (a - 1)) + (3 \times b))$$

Evaluate Arithmetic Expressions

- recursive method returning the value of a subtree
- when visiting an internal node, combine the values of the subtrees

```
Algorithm evalExpr(v)
    if isExternal (v)
        return v.element ()
    else
        x ← evalExpr(leftChild (v))
        y ← evalExpr(rightChild (v))
        ◊ ← operator stored at v
        return x ◊ y
```

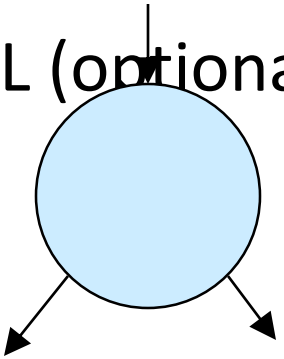


Binary Search Trees

- View today as data structures that can support dynamic set operations.
  - Search, Minimum, Maximum, Predecessor, Successor, Insert, and Delete.
- Can be used to build
  - Dictionaries.
  - Priority Queues.
- Basic operations take time proportional to the height of the tree – $O(h)$.

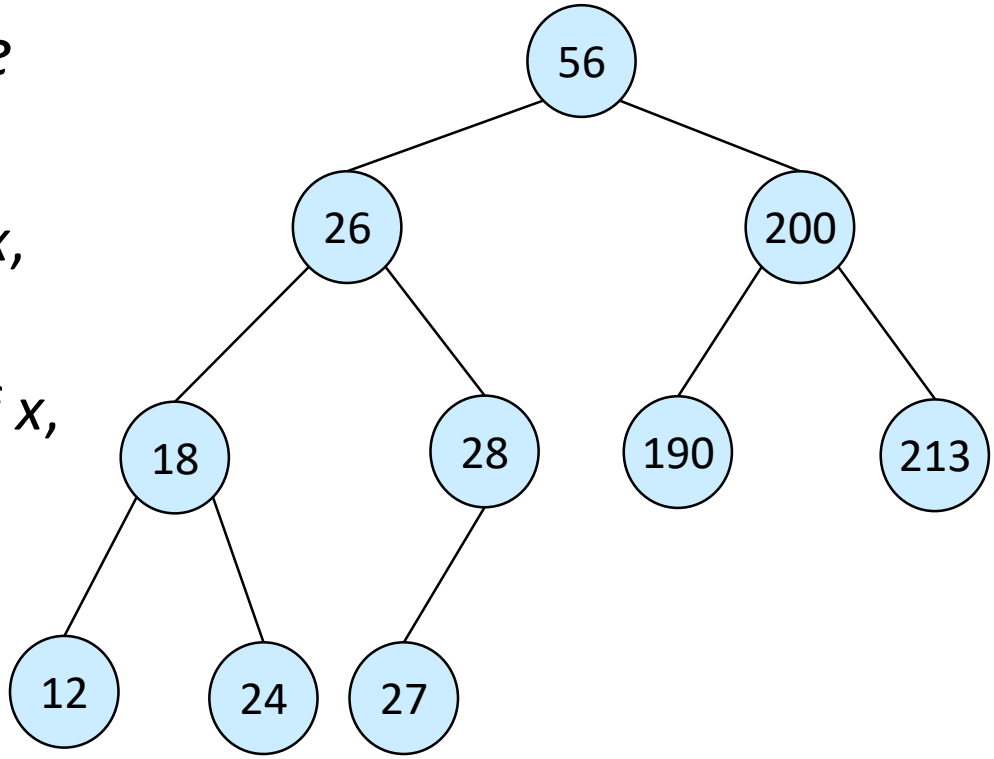
BST – Representation

- Represented by a linked data structure of nodes.
- *root*(*T*) points to the root of tree *T*.
- Each node contains fields:
  - *key*
  - *left* – pointer to left child: root of left subtree.
  - *right* – pointer to right child : root of right subtree.
  - *p* – pointer to parent. *p*[*root*[T]] = NIL (optional).



Binary Search Tree Property

- Stored keys must satisfy the *binary search tree* property.
  - $\forall$ *y* in left subtree of *x*, then $key[y] \leq key[x]$.
  - $\forall$ *y* in right subtree of *x*, then $key[y] \geq key[x]$.

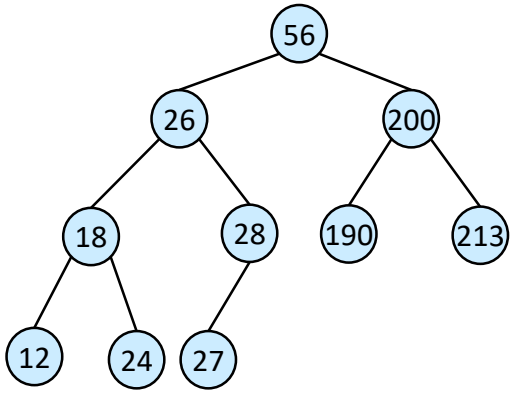


Inorder Traversal

The binary-search-tree property allows the keys of a binary search tree to be printed, in (monotonically increasing) order, recursively.

<u>Inorder-Tree-Walk (*x*)</u>

1. **if** $x \neq$ NIL
2. **then** Inorder-Tree-Walk(*left*[*p*])
3. print *key*[*x*]
4. Inorder-Tree-Walk(*right*[*p*])



- How long does the walk take?
- Can you prove its correctness?

Correctness of Inorder-Walk

- Must prove that it prints all elements, in order, and that it terminates.
- By induction on size of tree. Size=0: Easy.
- Size >1:
  - Prints left subtree in order by induction.
  - Prints root, which comes after all elements in left subtree (still in order).
  - Prints right subtree in order (all elements come after root, so still in order).

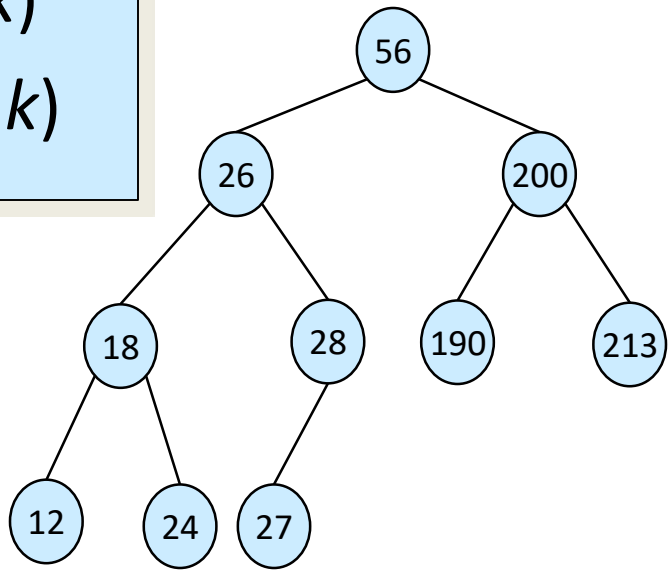
Querying a Binary Search Tree

- All dynamic-set search operations can be supported in $O(h)$ time.
- $h = \Theta(lg\ n)$ for a balanced binary tree (and for an average tree built by adding nodes in random order.)
- $h = \Theta(n)$ for an unbalanced tree that resembles a linear chain of n nodes in the worst case.

Tree Search

<u>Tree-Search(x, k)</u>

1. **if** x = NIL *or* k = *key*[x]
2. **then** return x
3. **if** k < *key*[x]
4. **then** return Tree-Search(*left*[x], k)
5. **else** return Tree-Search(*right*[x], k)



Running time: *O*(*h*)

Aside: tail-recursion

Predecessor and Successor

- Successor of node *x* is the node *y* such that *key*[*y*] is the smallest key greater than *key*[*x*].
- The successor of the largest key is NIL.
- Search consists of two cases.
  - If node *x* has a non-empty right subtree, then *x*'s successor is the minimum in the right subtree of *x*.
  - If node *x* has an empty right subtree, then:
    - As long as we move to the left up the tree (move up through right children), we are visiting smaller keys.
    - *x*'s successor *y* is the node that *x* is the predecessor of (*x* is the maximum in *y*'s left subtree).
    - In other words, *x*'s successor *y*, is the lowest ancestor of *x* whose left child is also an ancestor of *x*.

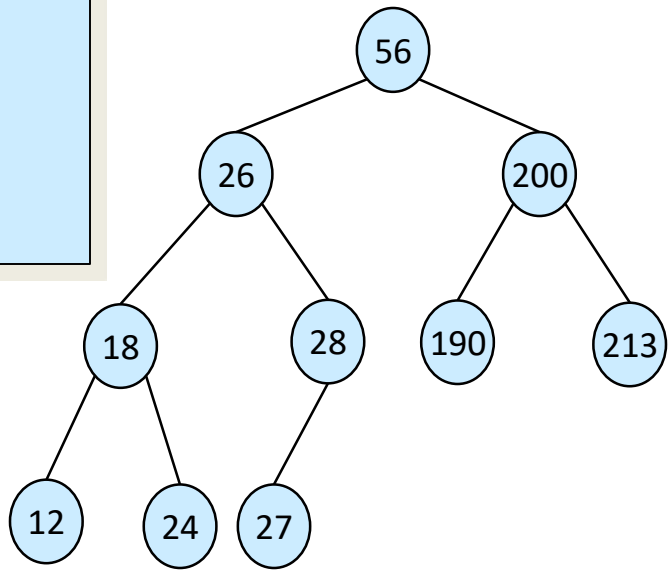
Pseudo-code for Successor

Tree-Successor(x)

```
if right[x] ≠ NIL
    then return Tree-Minimum(right[x])
y ← p[x]
while y ≠ NIL and x = right[y]
do x ← y
    y ← p[y]
return y
```

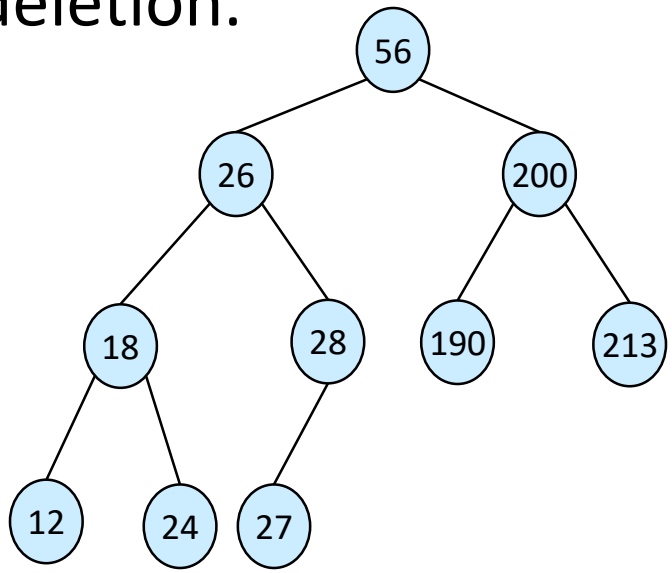
Code for ***predecessor*** is symmetric.

Running time: $O(h)$



BST Insertion – Pseudocode

- Change the dynamic set represented by a BST.
- Ensure the binary-search-tree property holds after change.
- Insertion is easier than deletion.




```
Tree-Insert(T, z)
y ← NIL
x ← root[T]
while x ≠ NIL
    do y ← x
        if key[z] < key[x]
            then x ← left[x]
            else x ← right[x]
p[z] ← y
if y = NIL
    then root[t] ← z
    else if key[z] < key[y]
        then  left[y] ← z
        else right[y] ← z
```

Analysis of Insertion

- Initialization: $O(1)$
- While loop in lines 3-7 searches for place to insert z, maintaining parent y.
  This takes $O(h)$ time.
- Lines 8-13 insert the value: $O(1)$

$\Rightarrow$ TOTAL: $O(h)$ time to insert a node.

```
Tree-Insert(T, z)
1.    y ← NIL
2.    x ← root[T]
3.    while x ≠ NIL
4.        do y ← x
5.            if key[z] < key[x]
6.                then x ← left[x]
7.                else x ← right[x]
8.    p[z] ← y
9.    if y = NIL
10.       then root[t] ← z
11.       else if key[z] < key[y]
12.           then  left[y] ← z
13.           else right[y] ← z
```

Exercise: Sorting Using BSTs

Sort (A)

for $i \leftarrow 1$ to n

do tree-insert($A[i]$)

inorder-tree-walk(*root*)

- What are the worst case and best case running times?
- In practice, how would this compare to other sorting algorithms?

Tree-Delete (T, x)

if x has no children ♦ case 0

then remove x

if x has one child ♦ case 1

then make $p[x]$ point to child

if x has two children (subtrees) ♦ case 2

then swap x with its successor

perform case 0 or case 1 to delete it

$\Rightarrow$ TOTAL: $O(h)$ time to delete a node

Deletion – Pseudocode

<u>Tree-Delete(*T*, *z*)</u>

/* Determine which node to splice out: either *z* or *z*'s successor. */

- **if** *left*[*z*] = NIL **or** *right*[*z*] = NIL
- **then** $y \leftarrow z$
- **else** $y \leftarrow$ Tree-Successor[z]

/* Set *x* to a non-NIL child of *x*, or to NIL if *y* has no children. */

4. **if** *left*[*y*] $\neq$ NIL

5. **then** $x \leftarrow$ *left*[*y*]

6. **else** $x \leftarrow$ *right*[*y*]

/* *y* is removed from the tree by manipulating pointers of *p*[*y*] and *x* */

7. **if** $x \neq$ NIL

8. **then** $p[x] \leftarrow p[y]$

/* Continued on next slide */

Deletion – Pseudocode

Tree-Delete(T, z) (Contd. from previous slide)

```
if p[y] = NIL
    then root[T] ← x
    else if  y ← left[p[i]]
            then left[p[y]] ← x
            else right[p[y]] ← x
/* If z's successor was spliced out, copy its data into z */
if y ≠ z
    then  key[z] ← key[y]
            copy y's satellite data into z.
return y
```

Binary Search Trees

- View today as data structures that can support dynamic set operations.
  - Search, Minimum, Maximum, Predecessor, Successor, Insert, and Delete.
- Can be used to build
  - Dictionaries.
  - Priority Queues.
- Basic operations take time proportional to the height of the tree – $O(h)$.

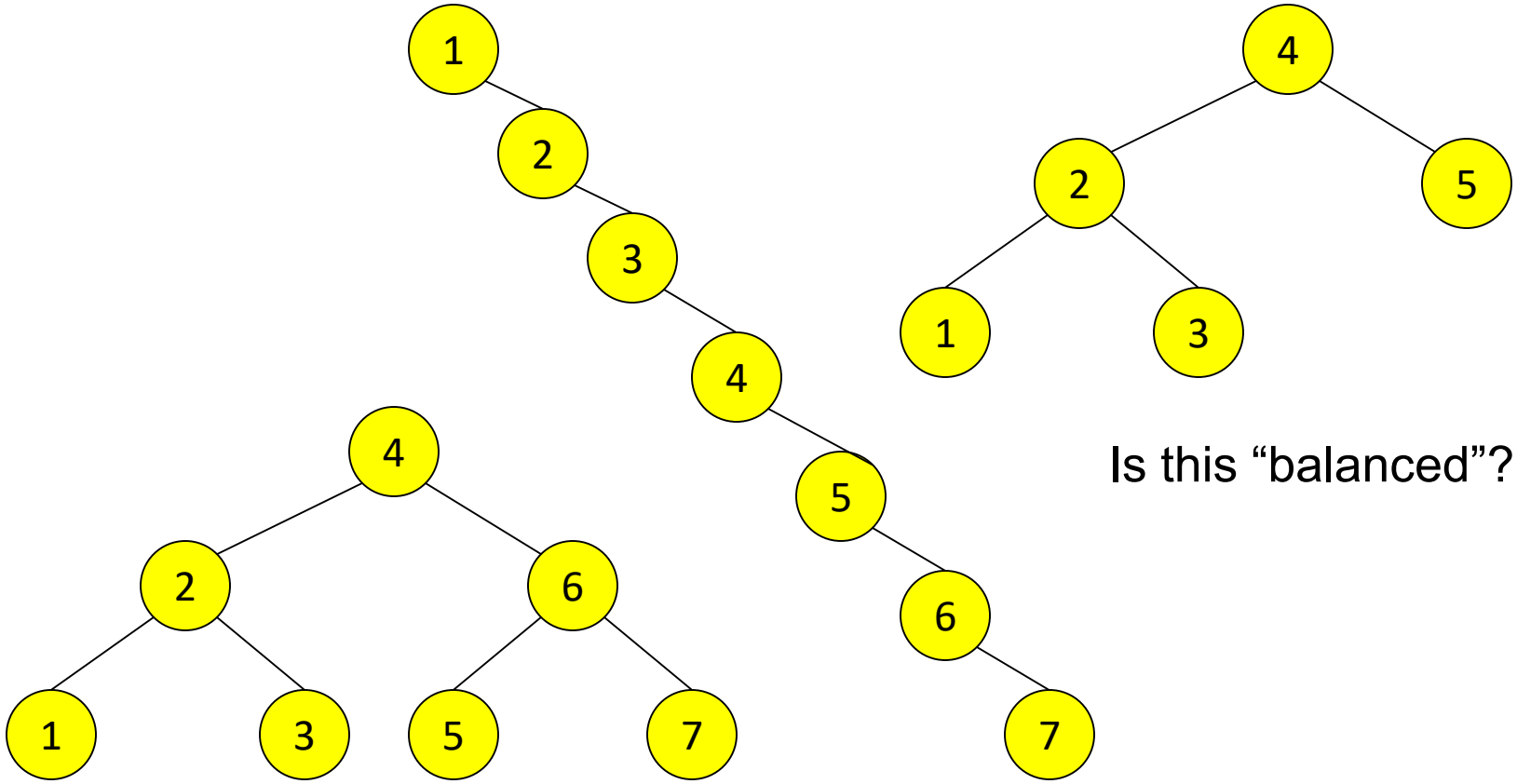
Binary Search Tree - Best Time

- All BST operations are O(d), where d is tree depth
- minimum d is $d = \lfloor \log_2 N \rfloor$ for a binary tree with N nodes
  - What is the best case tree?
  - What is the worst case tree?
- So, best case running time of BST operations is O(log N)

Binary Search Tree - Worst Time

- Worst case running time is O(N)
  - What happens when you Insert elements in ascending order?
    - Insert: 2, 4, 6, 8, 10, 12 into an empty BST
  - Problem: Lack of “balance”:
    - compare depths of left and right subtree
  - Unbalanced degenerate tree

Balanced and unbalanced BST



Is this “balanced”?

Approaches to balancing trees

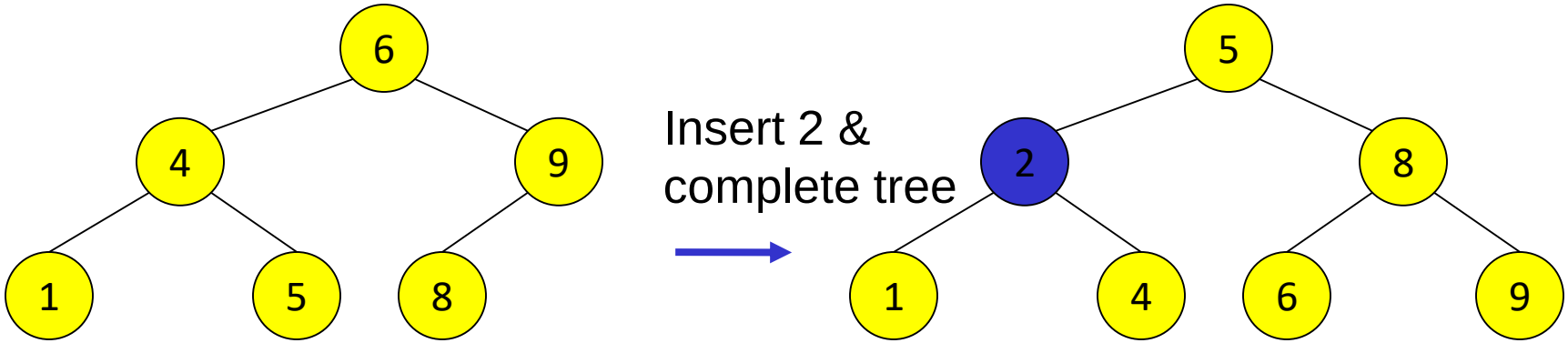
- Don't balance
  - May end up with some nodes very deep
- Strict balance
  - The tree must always be balanced perfectly
- Pretty good balance
  - Only allow a little out of balance
- Adjust on access
  - Self-adjusting

Balancing Binary Search Trees

- Many algorithms exist for keeping binary search trees balanced
  - Adelson-Velskii and Landis (AVL) trees (height-balanced trees)
  - Splay trees and other self-adjusting trees
  - B-trees and other multiway search trees

Perfect Balance

- Want a complete tree after every operation
  - tree is full except possibly in the lower right
- This is expensive
  - For example, insert 2 in the tree on the left and then rebuild as a complete tree

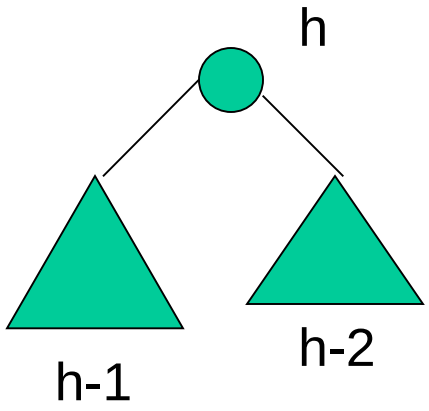


AVL - Good but not Perfect Balance

- AVL trees are height-balanced binary search trees
- Balance factor of a node
  - height(left subtree) - height(right subtree)
- An AVL tree has balance factor calculated at every node
  - For every node, heights of left and right subtree can differ by no more than 1
  - Store current heights in each node

Height of an AVL Tree

- N(h) = minimum number of nodes in an AVL tree of height h.
- Basis
  - N(0) = 1, N(1) = 2
- Induction
  - N(h) = N(h-1) + N(h-2) + 1
- Solution (recall Fibonacci analysis)
  - $N(h) \geq \phi^h \quad (\phi \approx 1.62)$



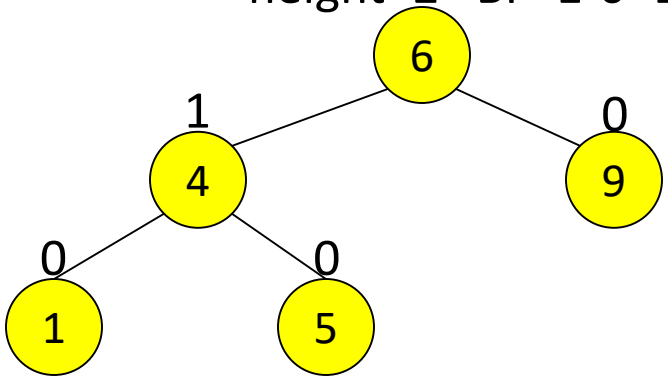
Height of an AVL Tree

- $N(h) \geq \phi^h \quad (\phi \approx 1.62)$
- Suppose we have n nodes in an AVL tree of height h.
  - $n \geq N(h)$ (because N(h) was the minimum)
  - $n \geq \phi^h$ hence $\log_\phi n \geq h$ (relatively well balanced tree!!)
  - $h \leq 1.44 \log_2 n$ (i.e., Find takes O(logn))

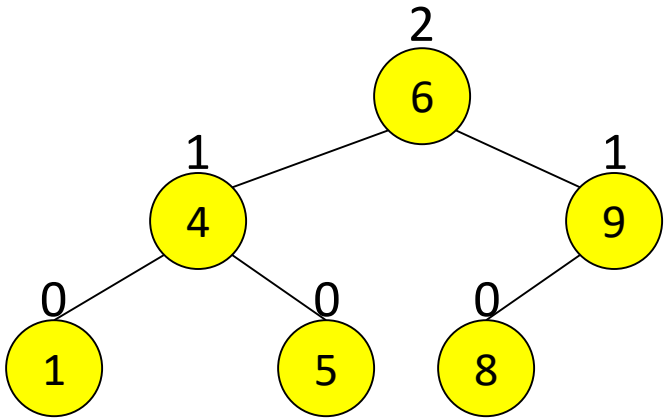
Node Heights

Tree A (AVL)

height=2 BF=1-0=1



Tree B (AVL)



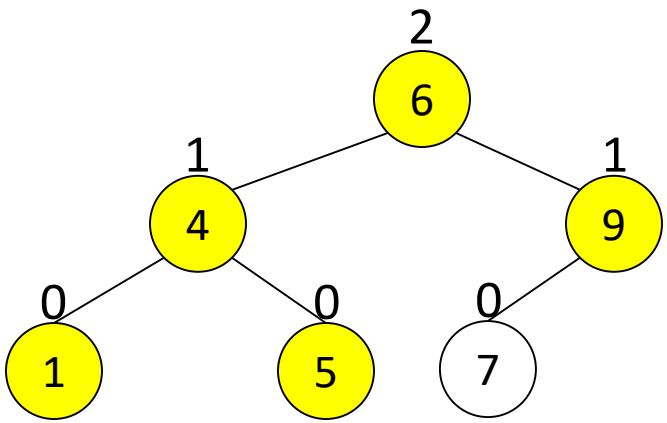
height of node = h

balance factor = h_{left}-h_{right}

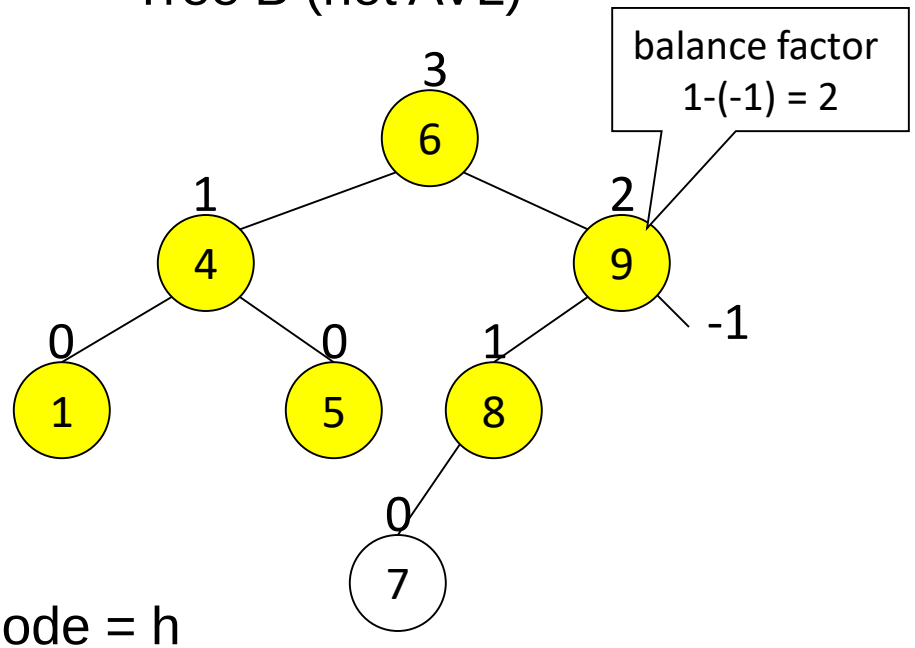
empty height = -1

Node Heights after Insert 7

Tree A (AVL)



Tree B (not AVL)

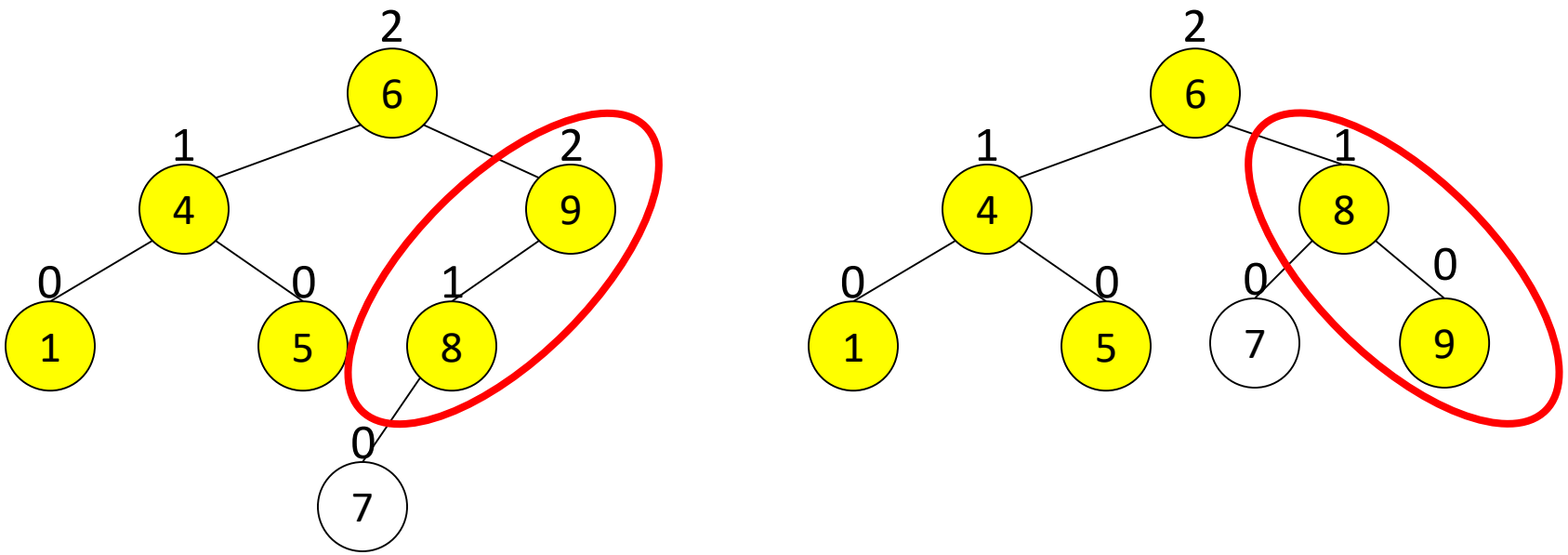


height of node = h
balance factor = h_{left}-h_{right}
empty height = -1

Insert and Rotation in AVL Trees

- Insert operation may cause balance factor to become 2 or –2 for some node
  - only nodes on the path from insertion point to root node have possibly changed in height
  - So after the Insert, go back up to the root node by node, updating heights
  - If a new balance factor (the difference h_{left}-h_{right}) is 2 or –2, adjust tree by *rotation* around the node

Single Rotation in an AVL Tree



Insertions in AVL Trees

Let the node that needs rebalancing be α.

There are 4 cases:

Outside Cases (require single rotation) :

1. Insertion into left subtree of left child of α.
2. Insertion into right subtree of right child of α.

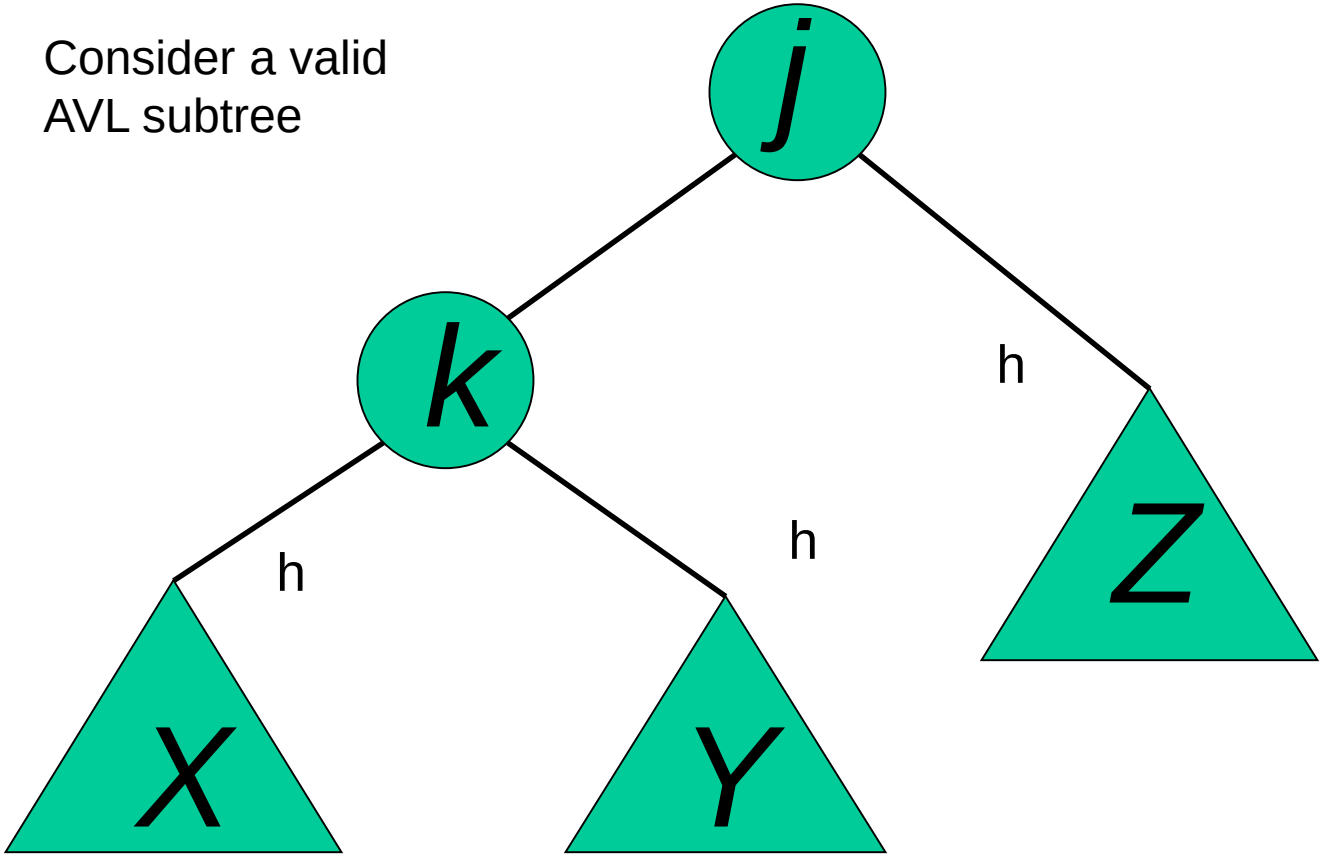
Inside Cases (require double rotation) :

3. Insertion into right subtree of left child of α.
4. Insertion into left subtree of right child of α.

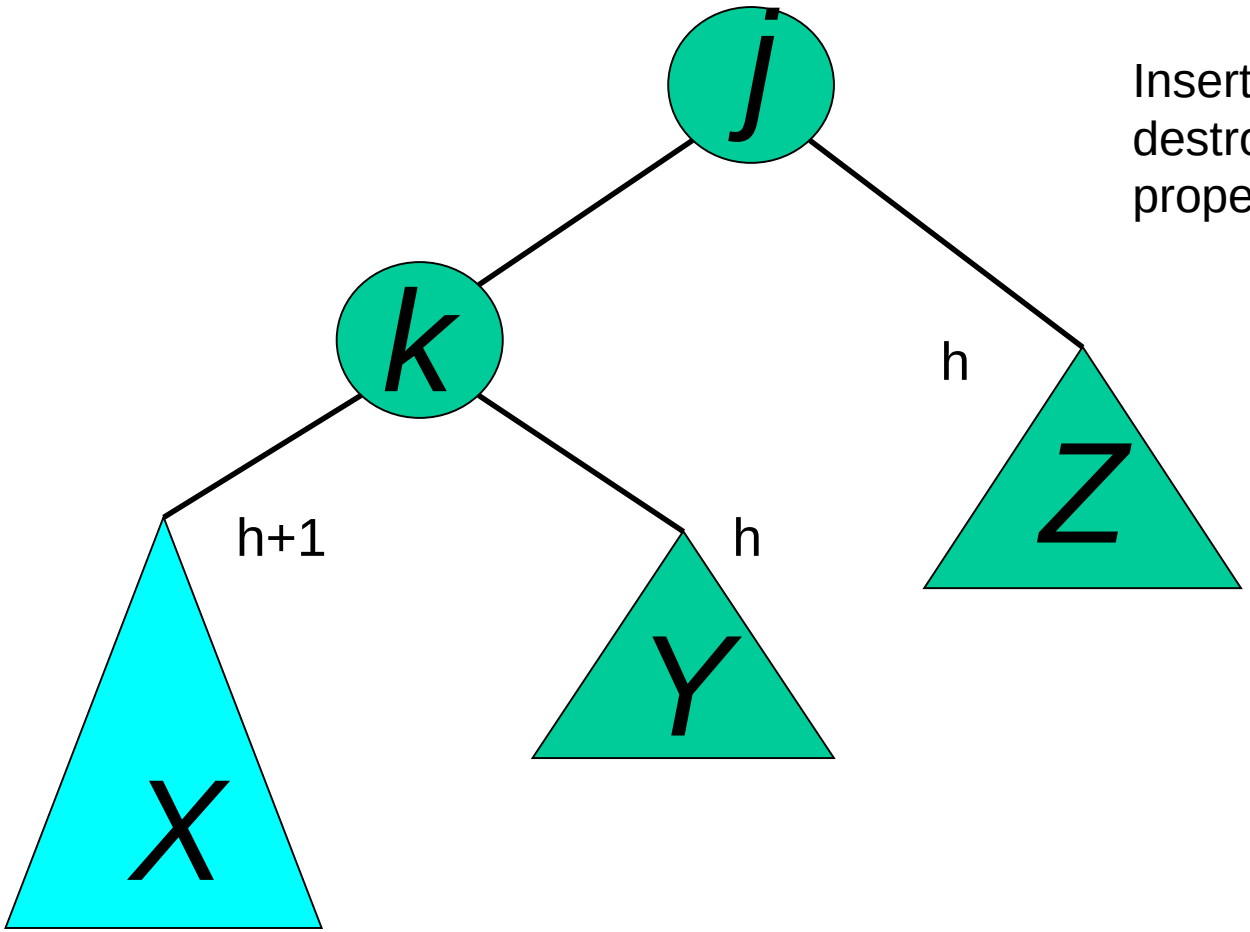
The rebalancing is performed through four separate rotation algorithms.

AVL Insertion: Outside Case

Consider a valid
AVL subtree

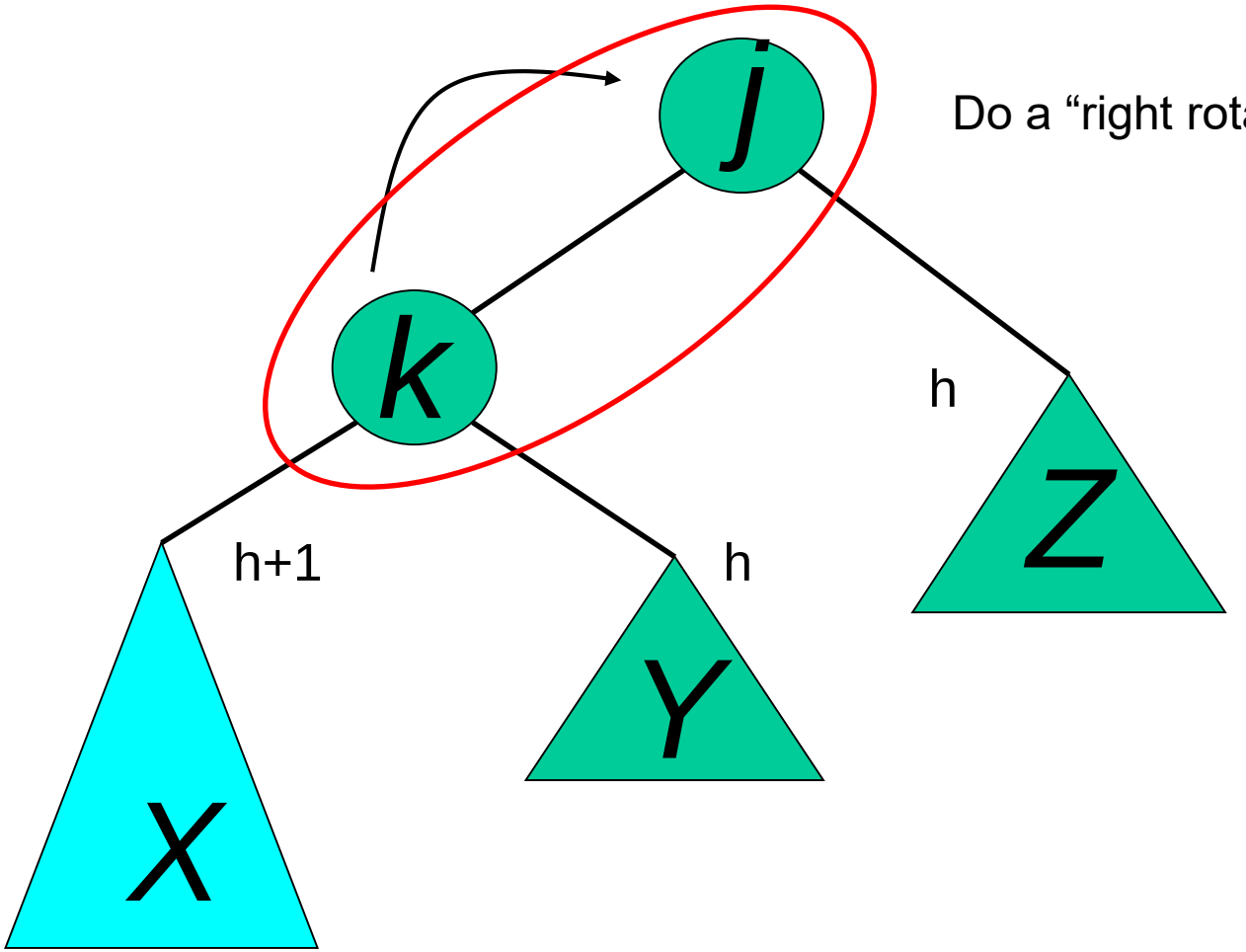


AVL Insertion: Outside Case



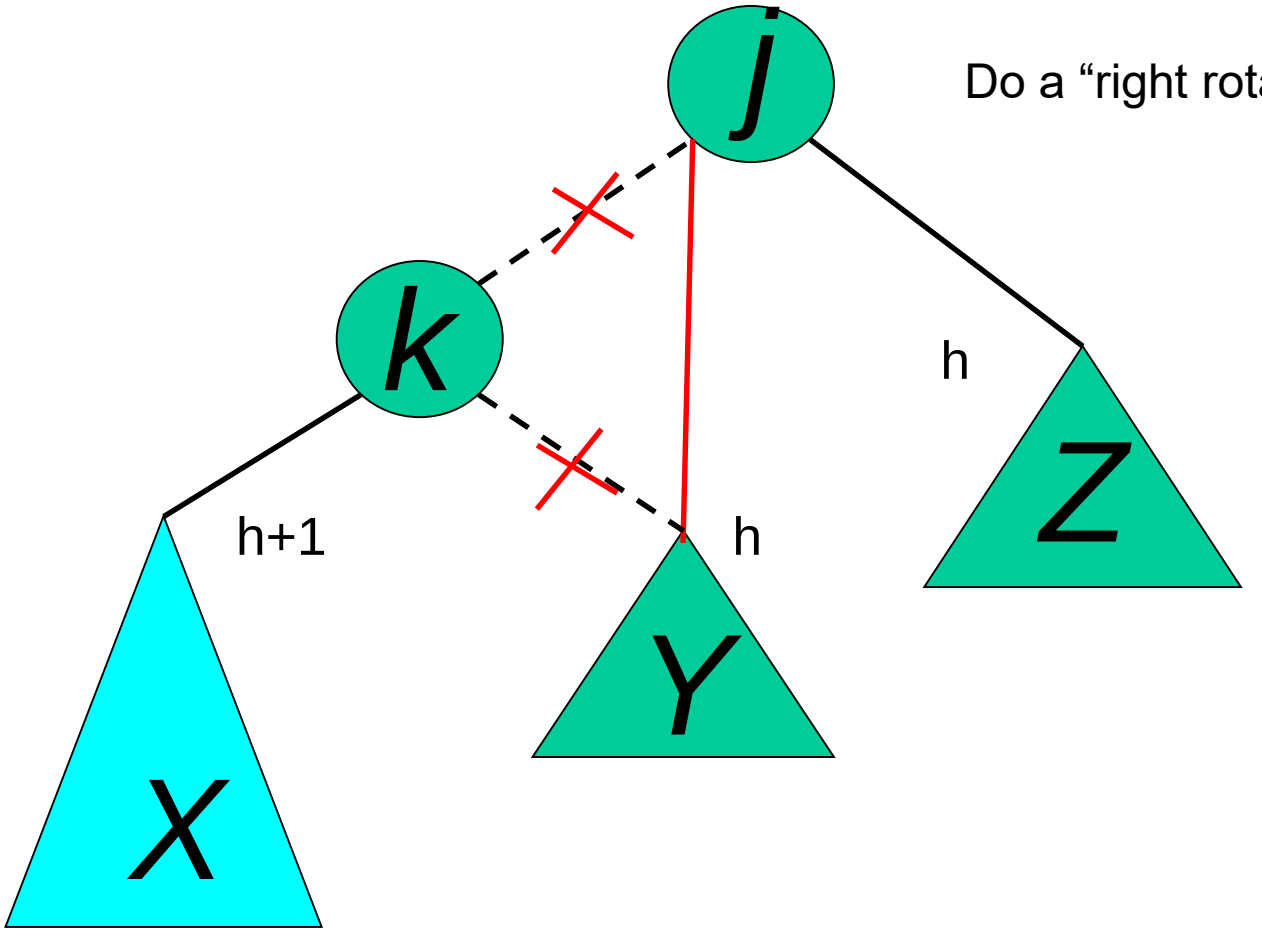
Inserting into X destroys the AVL property at node j

AVL Insertion: Outside Case



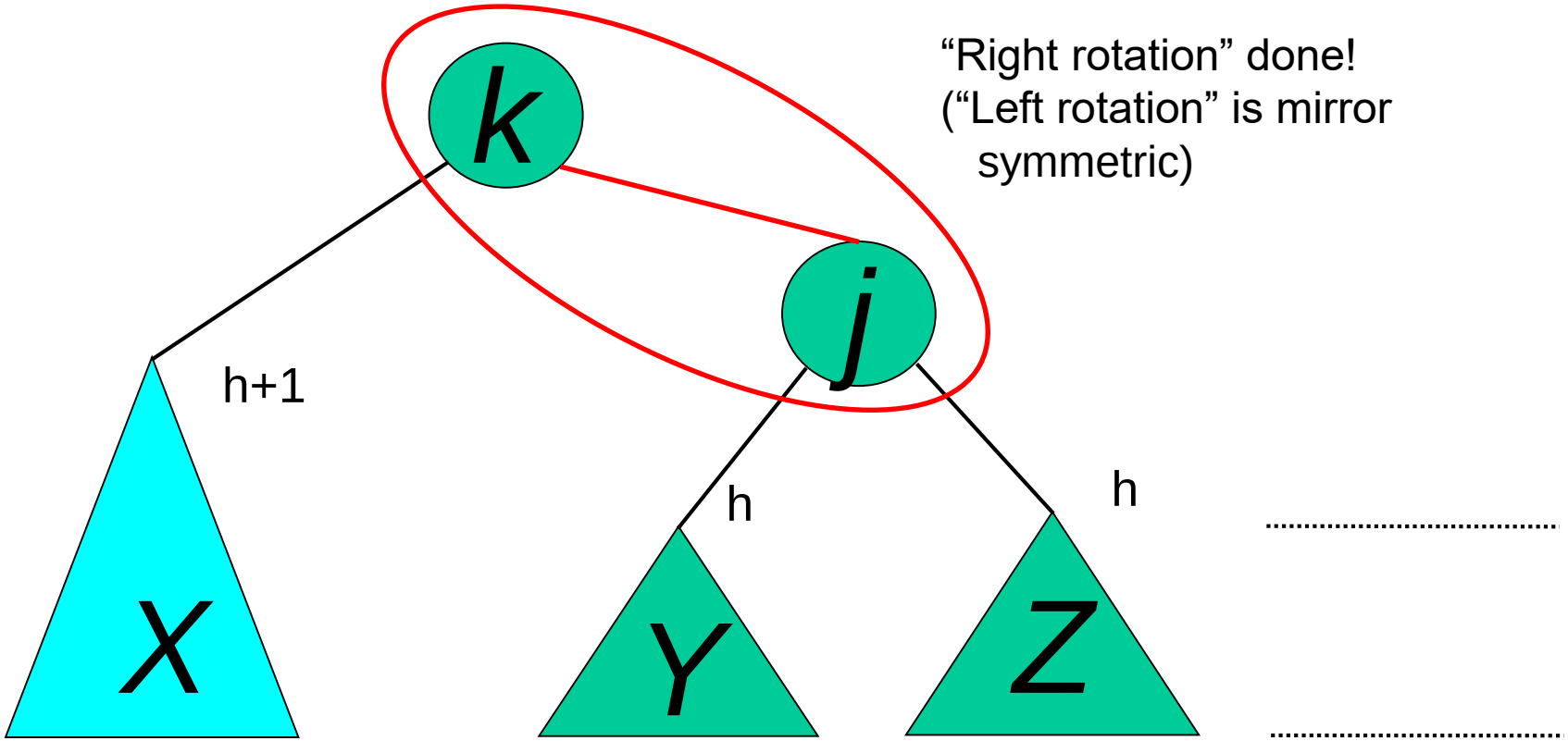
Do a “right rotation”

Single right rotation



Do a “right rotation”

Outside Case Completed

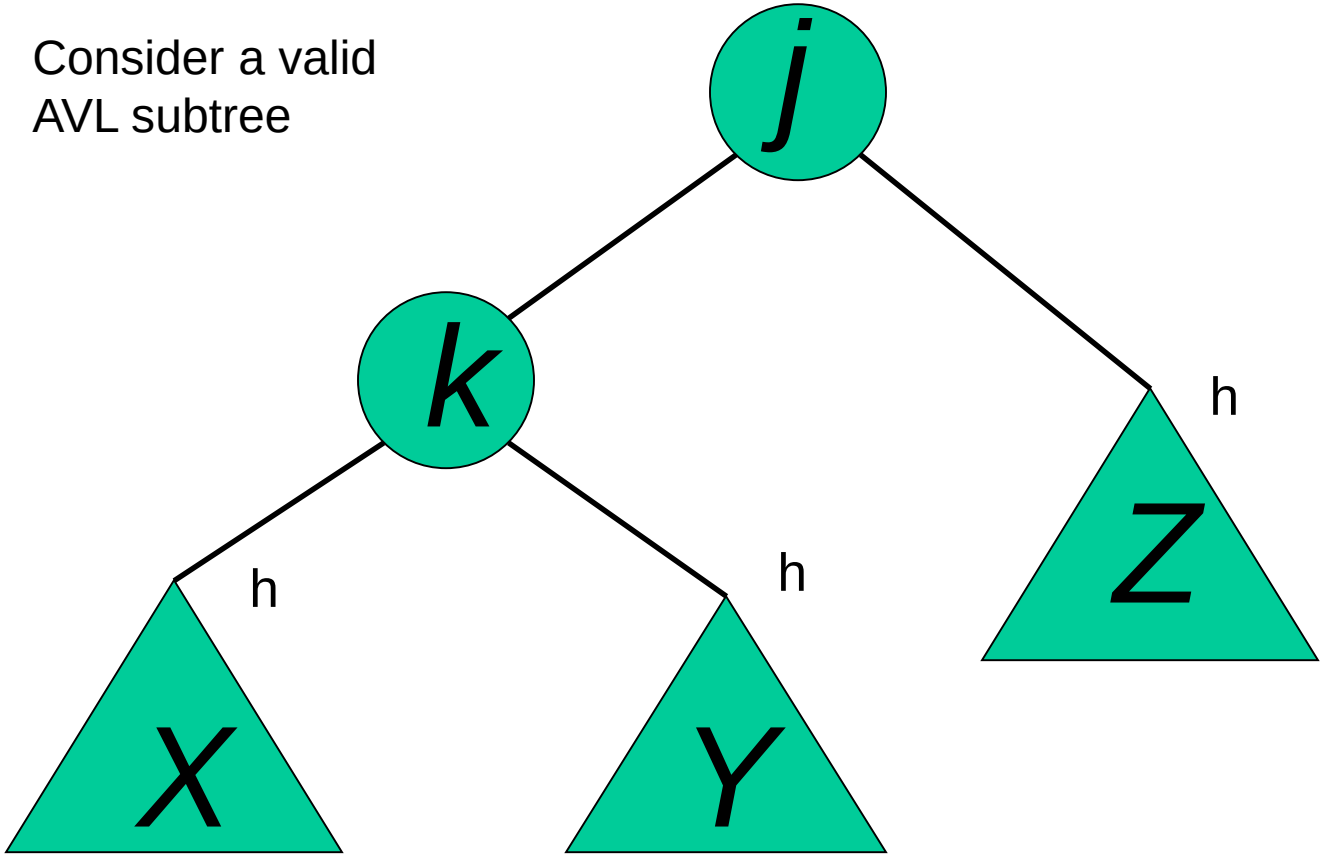


"Right rotation" done!
("Left rotation" is mirror symmetric)

AVL property has been restored!

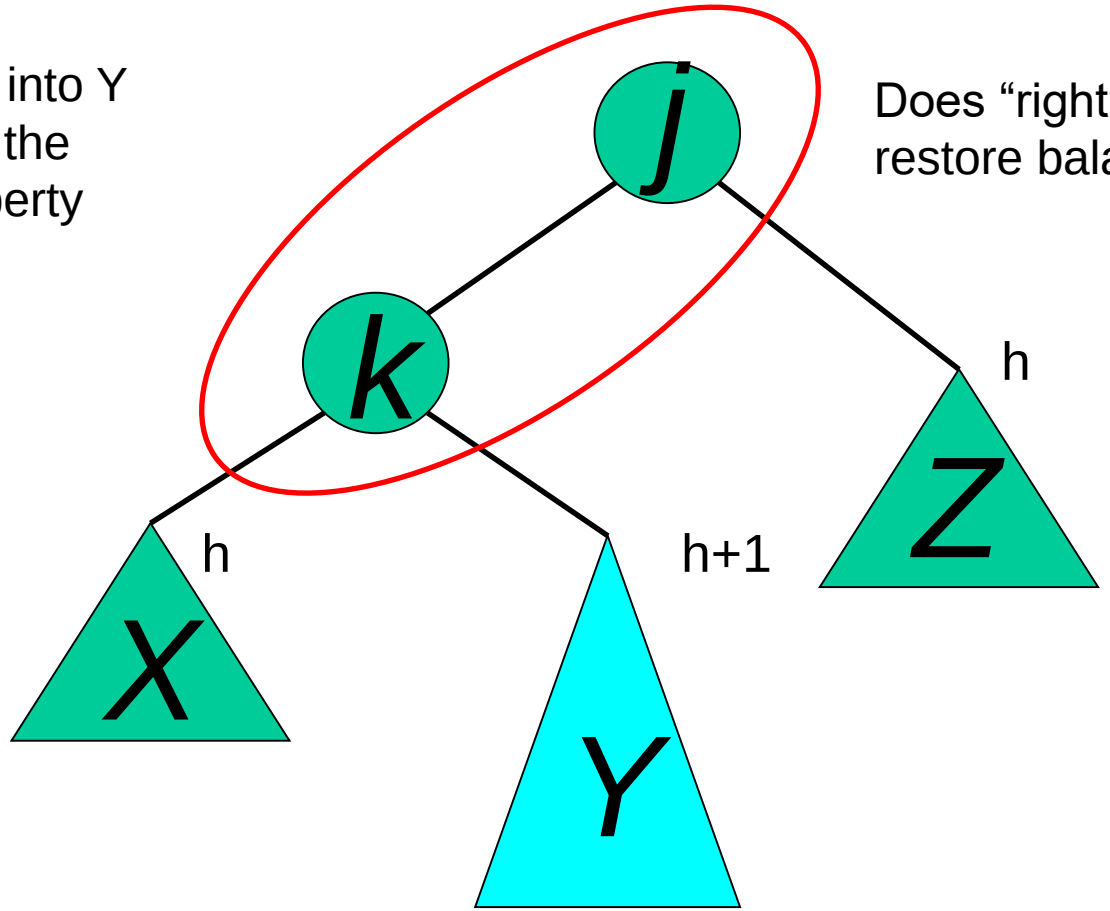
AVL Insertion: Inside Case

Consider a valid AVL subtree



AVL Insertion: Inside Case

Inserting into Y destroys the AVL property at node j



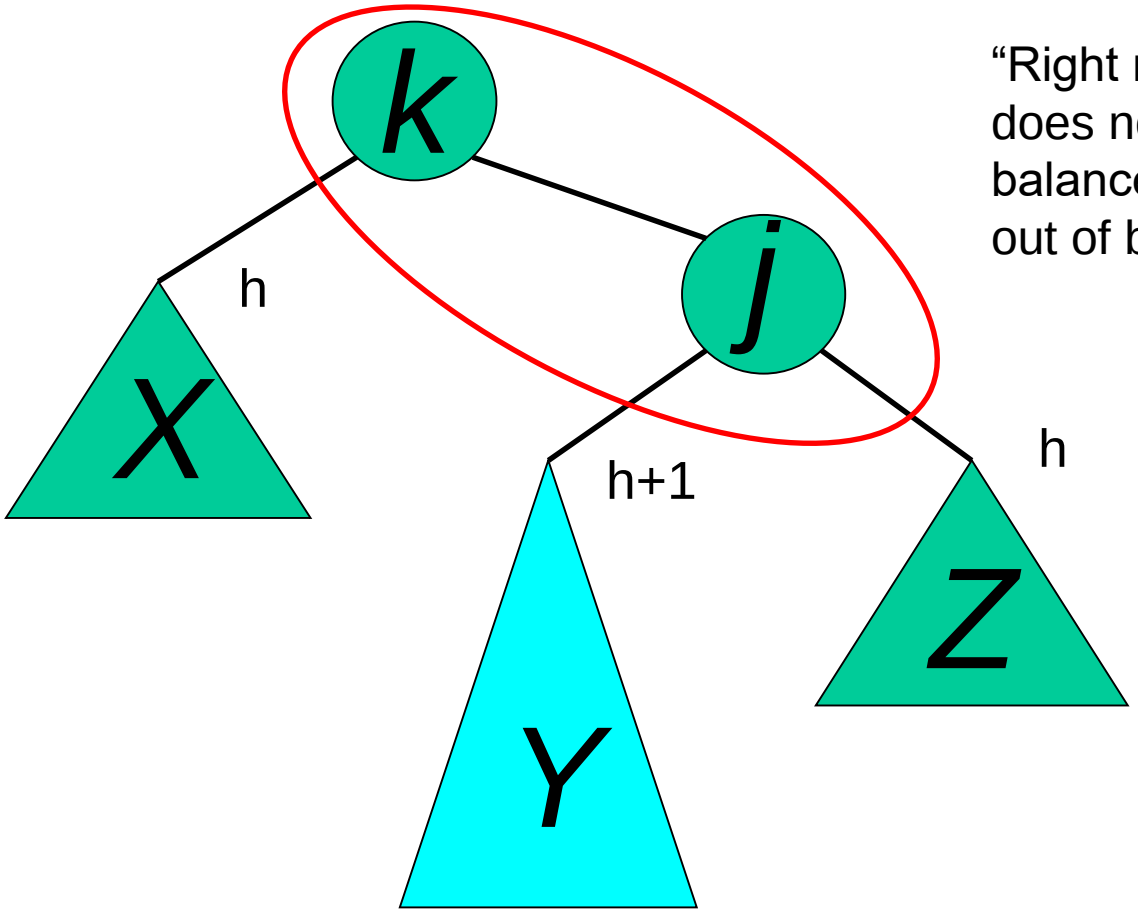
Does “right rotation” restore balance?

.....

.....

.....

AVL Insertion: Inside Case



"Right rotation" does not restore balance… now k is out of balance

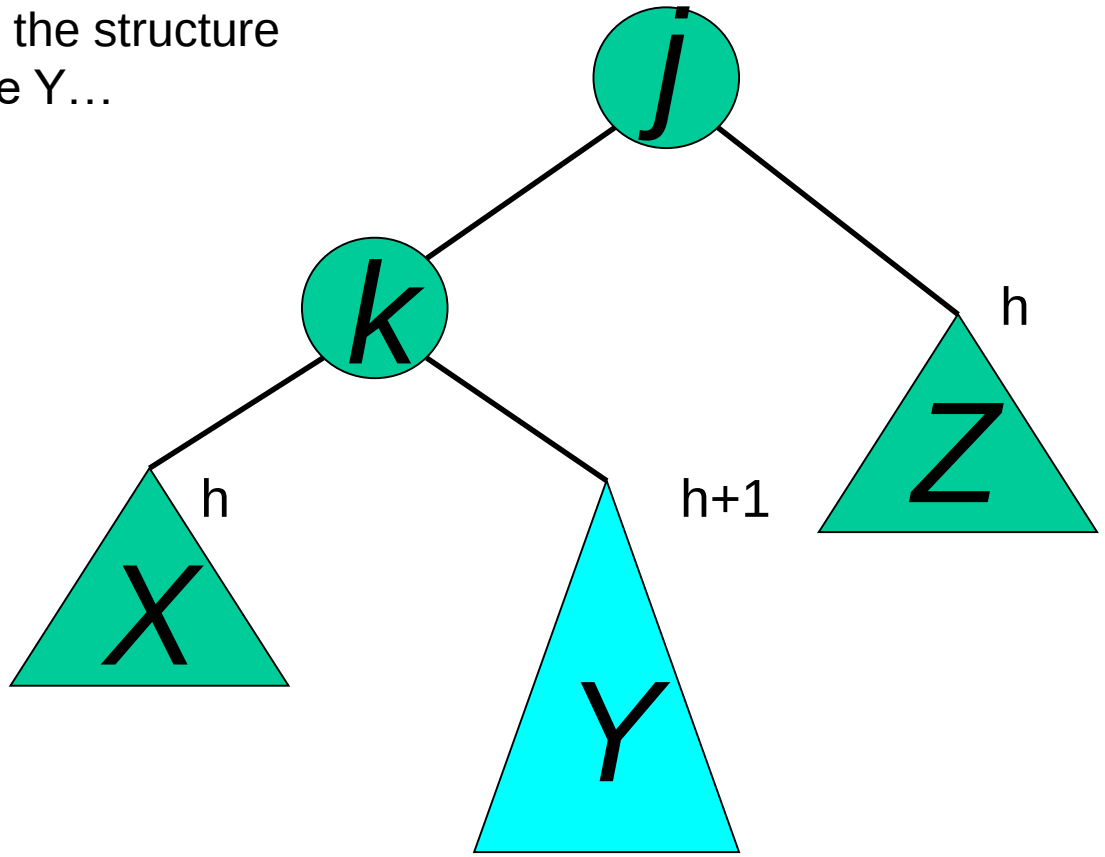
.....

.....

.....

AVL Insertion: Inside Case

Consider the structure of subtree Y…



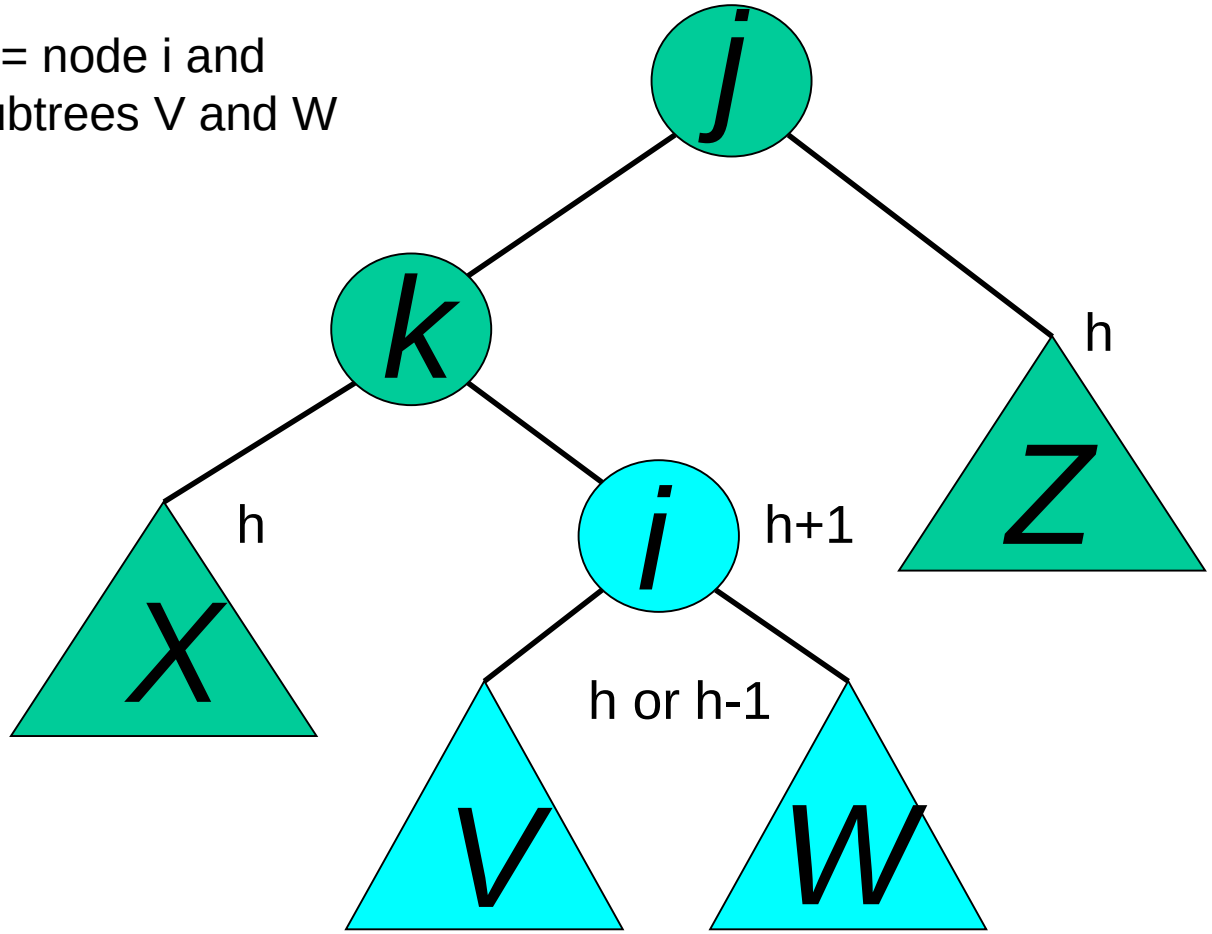
.....

.....

.....

AVL Insertion: Inside Case

Y = node i and
subtrees V and W

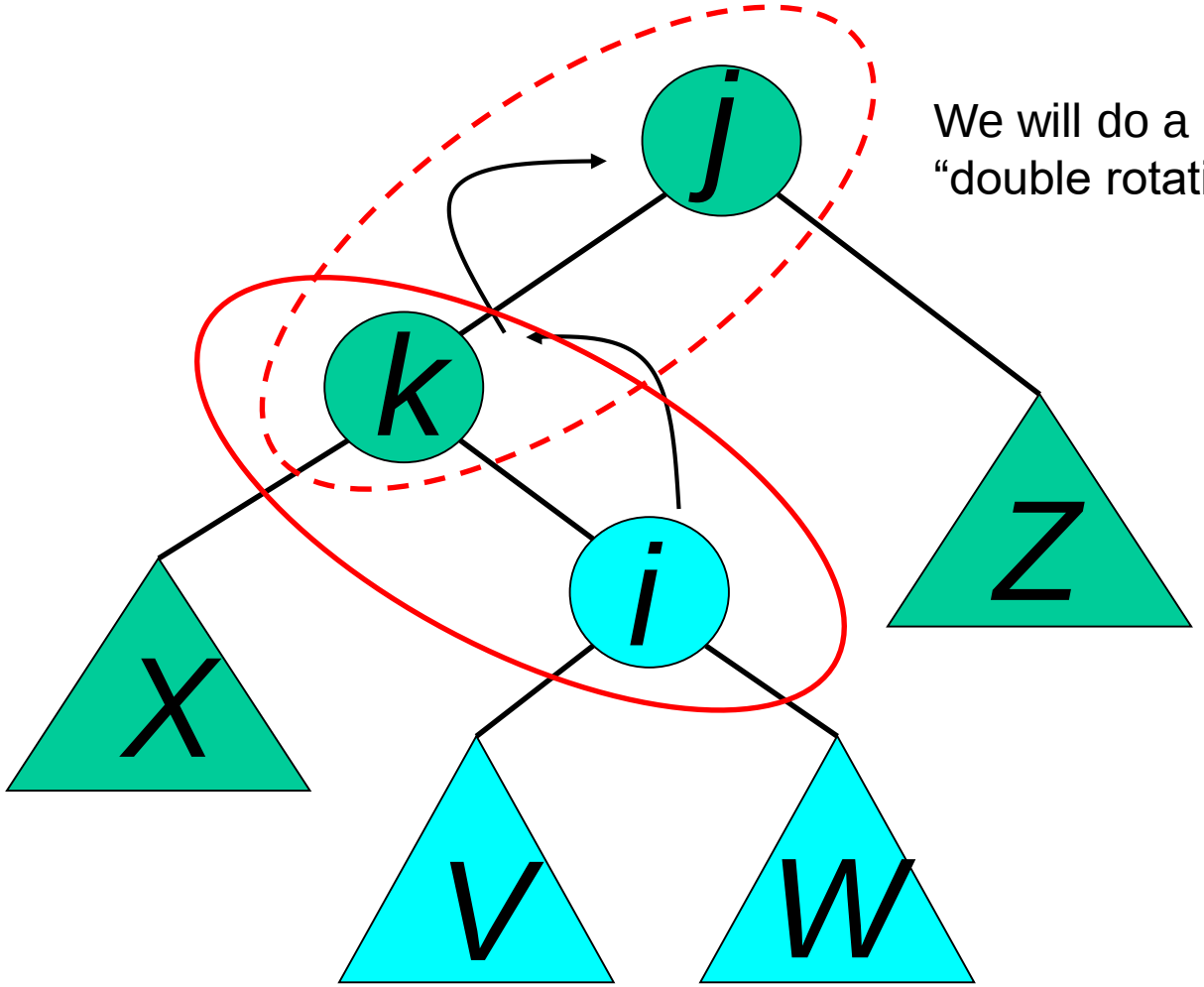


.....

.....

.....

AVL Insertion: Inside Case



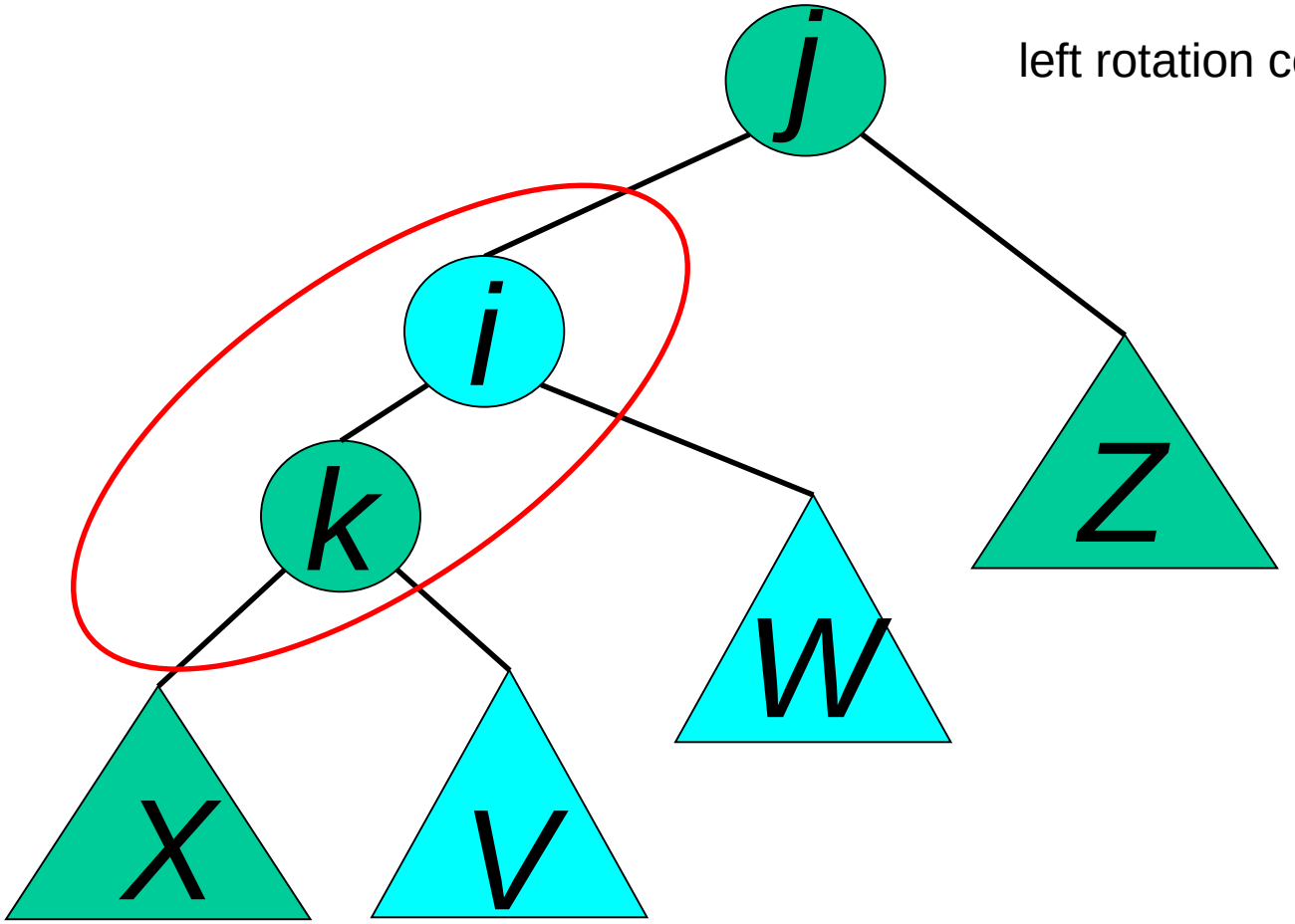
We will do a left-right "double rotation" . . .

.....

.....

.....

Double rotation : first rotation



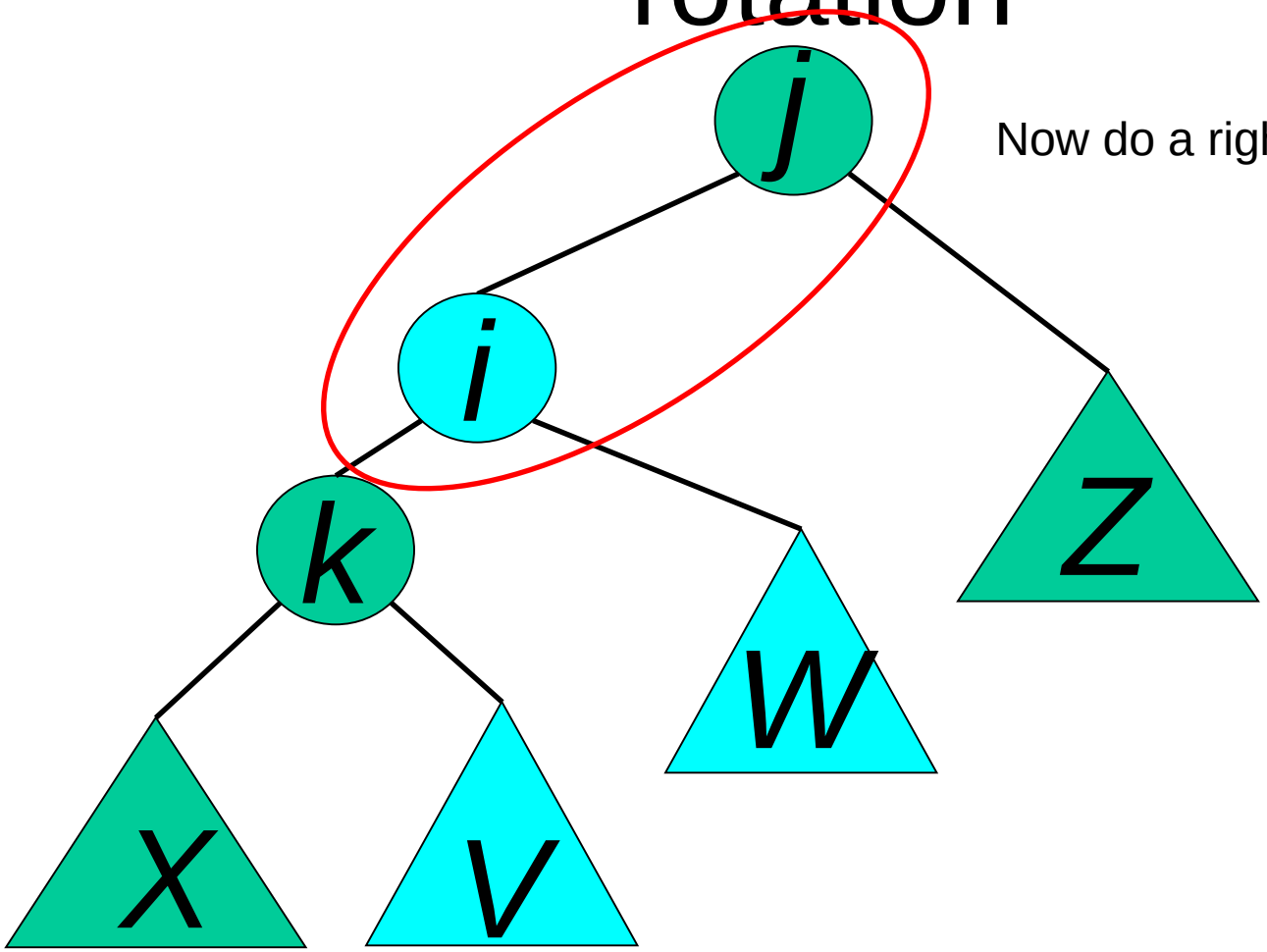
left rotation complete

.....

.....

.....

Double rotation : second rotation



Now do a right rotation

.....

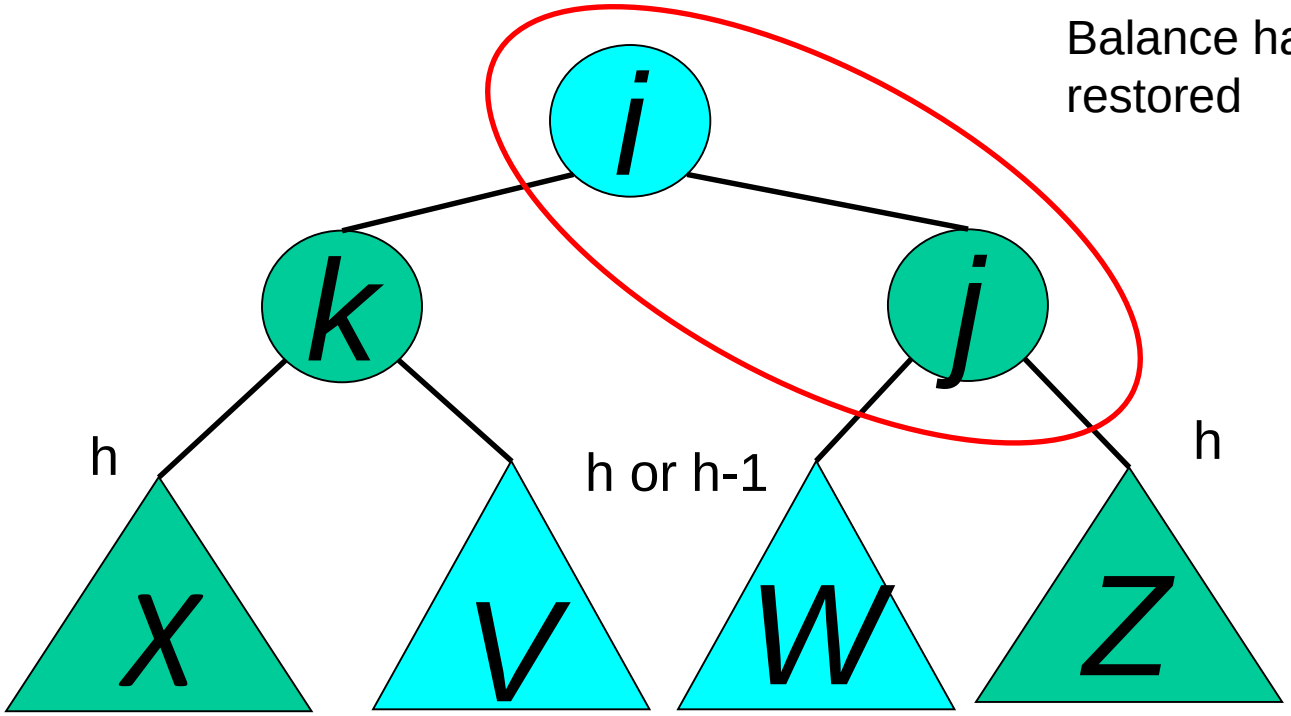
.....

.....

Double rotation : second rotation

right rotation complete

Balance has been restored

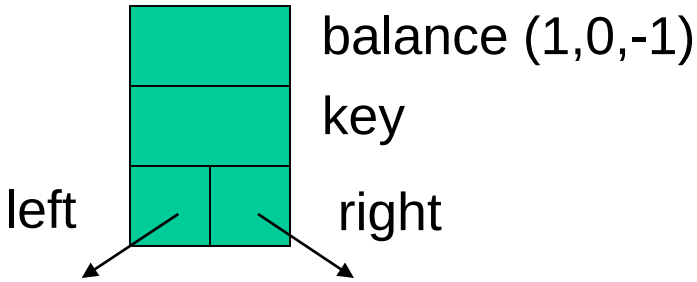


.....

.....

.....

Implementation



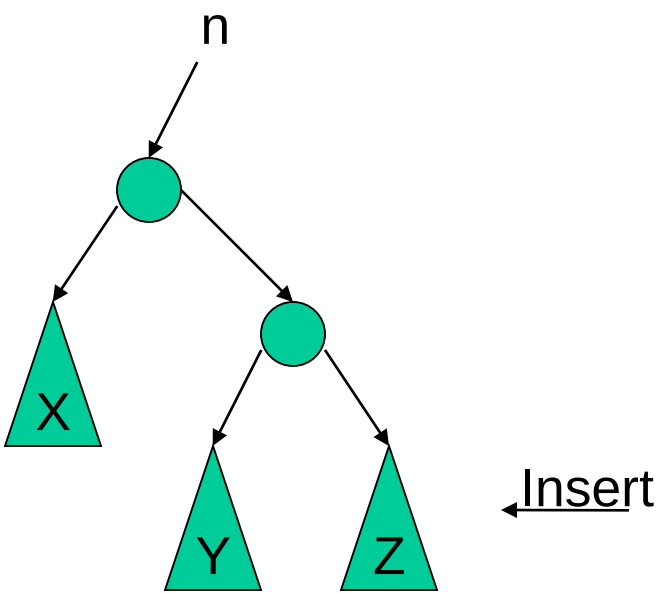
No need to keep the height; just the difference in height, i.e. the balance factor; this has to be modified on the path of insertion even if you don't perform rotations

Once you have performed a rotation (single or double) you won't need to go back up the tree

Single Rotation

```
RotateFromRight(n : reference node pointer) {
p : node pointer;
p := n.right;
n.right := p.left;
p.left := n;
n := p
}
```

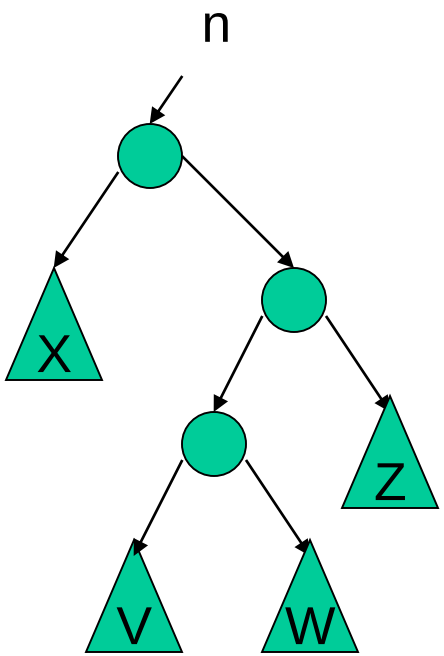
You also need to modify the heights or balance factors of n and p



Double Rotation

- Implement Double Rotation in two lines.

```
DoubleRotateFromRight(n : reference node pointer) {
????
}
```



Insertion in AVL Trees

- Insert at the leaf (as for all BST)
  - only nodes on the path from insertion point to root node have possibly changed in height
  - So after the Insert, go back up to the root node by node, updating heights
  - If a new balance factor (the difference h_{left}-h_{right}) is 2 or –2, adjust tree by *rotation* around the node

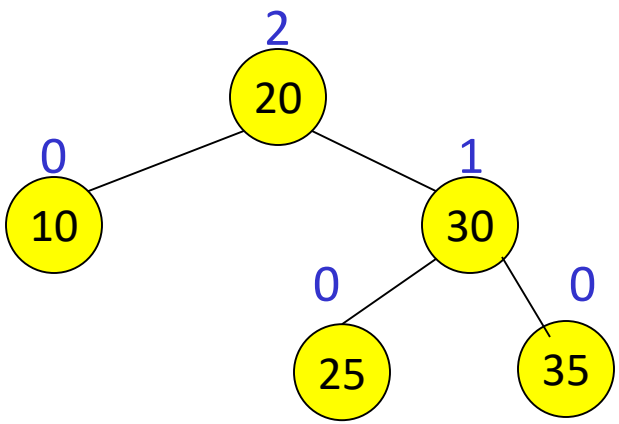
Insert in BST

```
Insert(T : reference tree pointer, x : element) : integer {
if T = null then
  T := new tree; T.data := x; return 1;//the links to
                                  //children are null
case
  T.data = x : return 0; //Duplicate do nothing
  T.data > x : return Insert(T.left, x);
  T.data < x : return Insert(T.right, x);
endcase
}
```

Insert in AVL trees

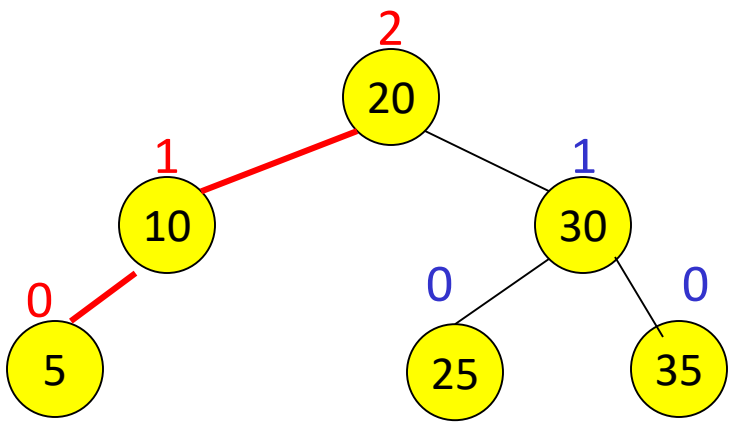
```
Insert(T : reference tree pointer, x : element) : {
if T = null then
  {T := new tree; T.data := x; height := 0; return;}
case
  T.data = x : return ; //Duplicate do nothing
  T.data > x : Insert(T.left, x);
               if ((height(T.left)- height(T.right)) = 2){
                  if (T.left.data > x ) then //outside case
                         T = RotatefromLeft (T);
                  else                        //inside case
                         T = DoubleRotatefromLeft (T);}
  T.data < x :  Insert(T.right, x);
                code similar to the left case
Endcase
  T.height := max(height(T.left),height(T.right)) +1;
  return;
}
```

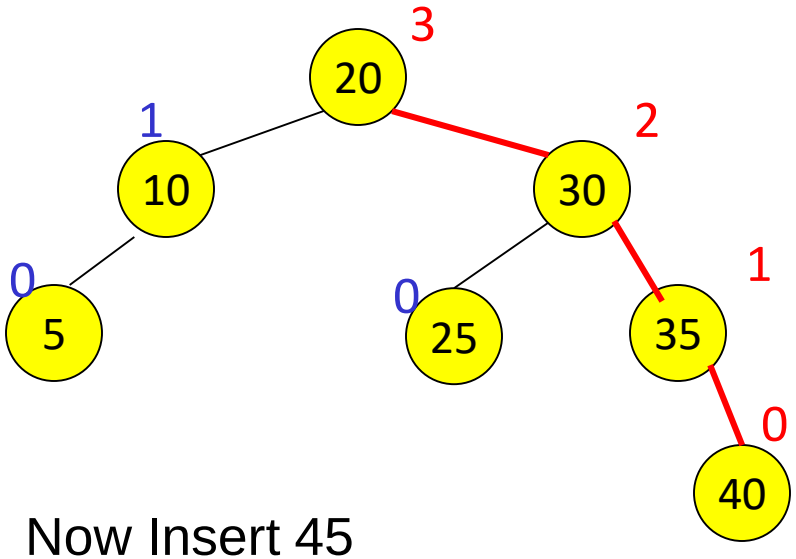
Example of Insertions in an AVL Tree



Insert 5, 40

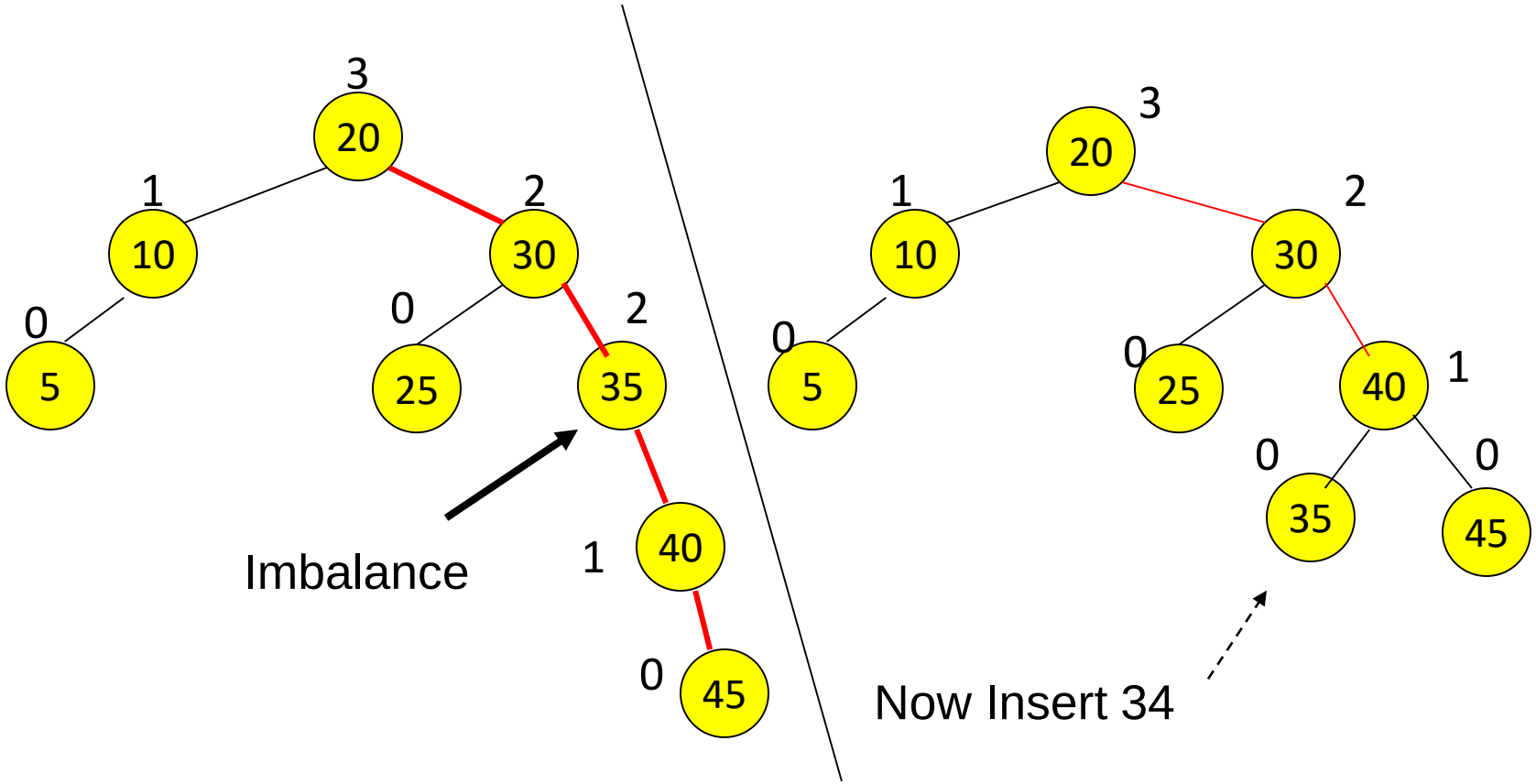
Example of Insertions in an AVL Tree



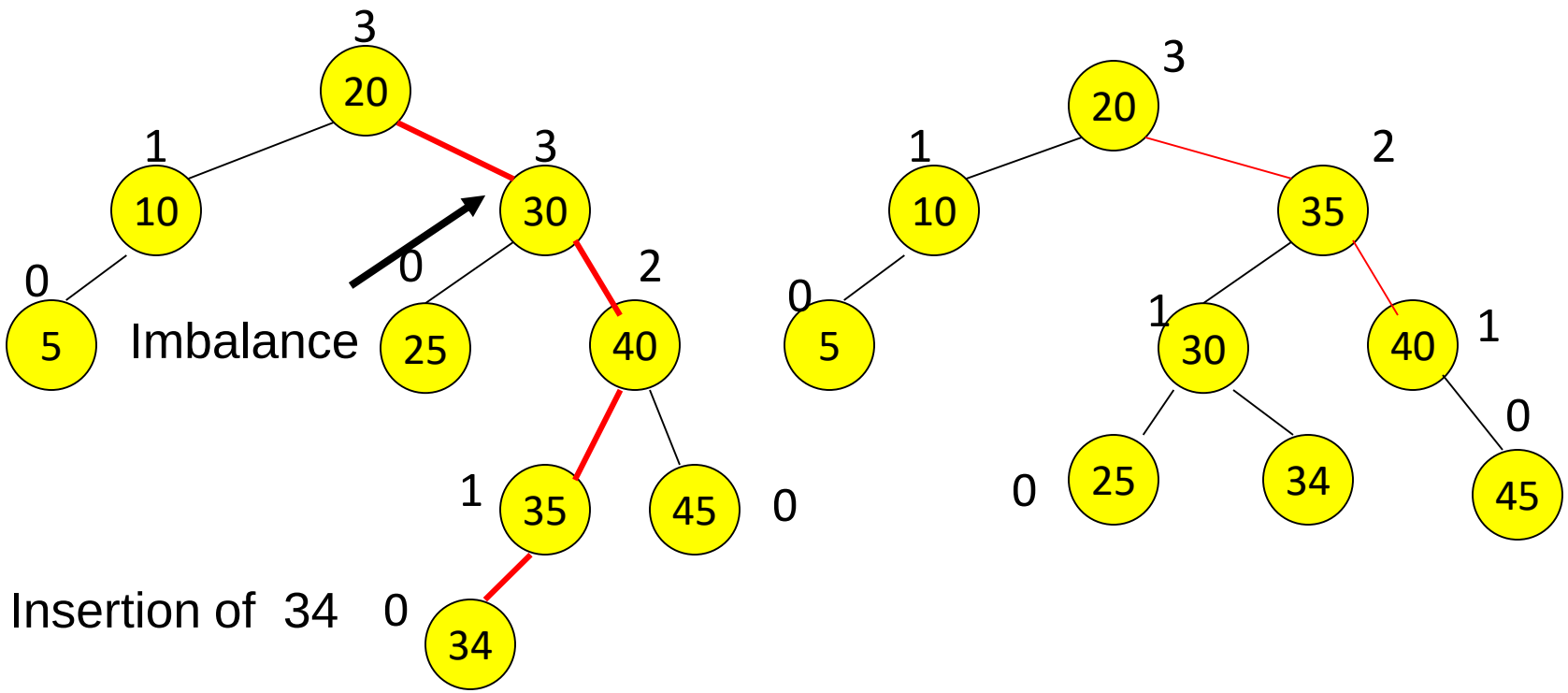


Now Insert 45

Single rotation (outside case)



Double rotation (inside case)



AVL Tree Deletion

- Similar but more complex than insertion
  - Rotations and double rotations needed to rebalance
  - Imbalance may propagate upward so that many rotations may be needed.

Pros and Cons of AVL Trees

Arguments for AVL trees:

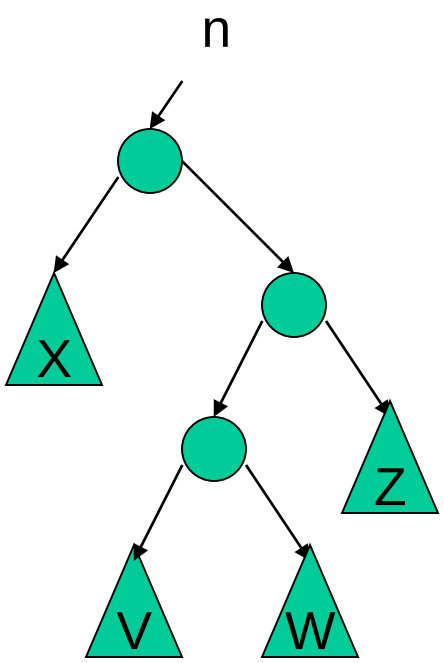
1. Search is O(log N) since AVL trees are always balanced.
2. Insertion and deletions are also O(logn)
3. The height balancing adds no more than a constant factor to the speed of insertion.

Arguments against using AVL trees:

1. Difficult to program & debug; more space for balance factor.
2. Asymptotically faster but rebalancing costs time.
3. Most large searches are done in database systems on disk and use other structures (e.g. B-trees).
4. May be OK to have O(N) for a single operation if total run time for many consecutive operations is fast (e.g. Splay trees).

Double Rotation Solution

```
DoubleRotateFromRight(n : reference node pointer) {
RotateFromLeft(n.right);
RotateFromRight(n);
}
```



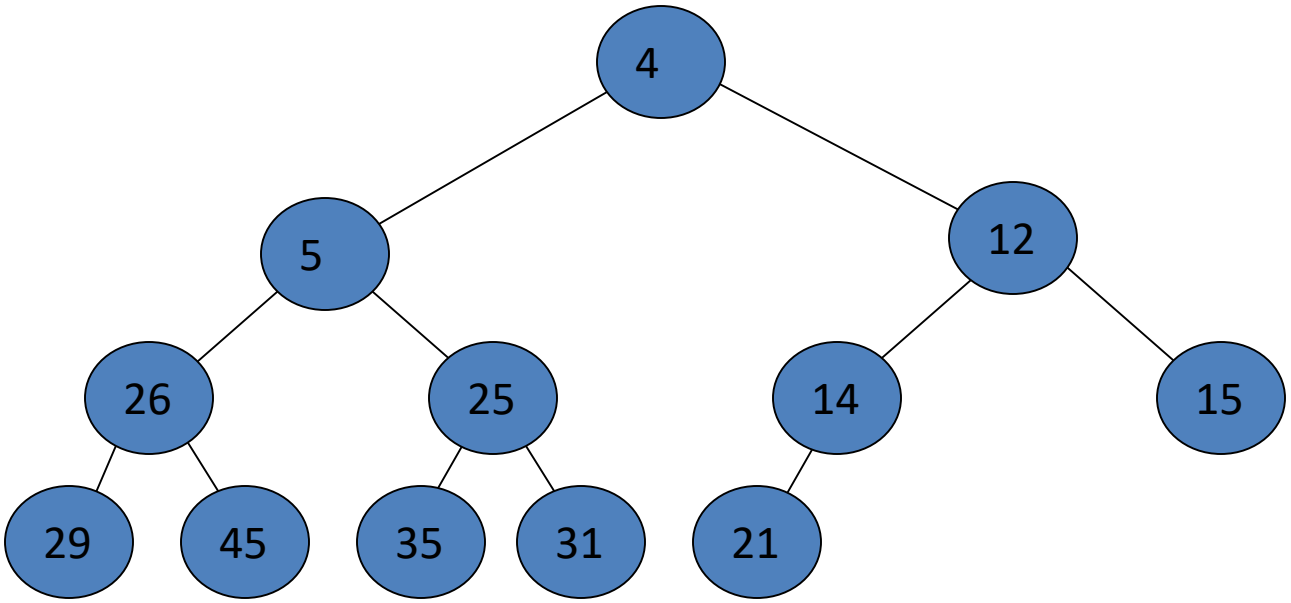
Binary Heaps
Motivation

- Development of a data structure which allows efficient inserts and efficient deletes of the minimum value (minheap) or maximum value (maxheap)

Implementation

- One-array implementation of a binary tree
- Root of tree is at element 1 of the array
- If a node is at element *i* of the array, then its children are at elements $2*i$ and $2*i+1$
- If a node is at element *i* of the array, then its parent is at element $floor(i/2)=\lfloor i/2 \rfloor$

Implementation



i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
array	__	4	5	12	26	25	14	15	29	45	35	31	21	__	__	__

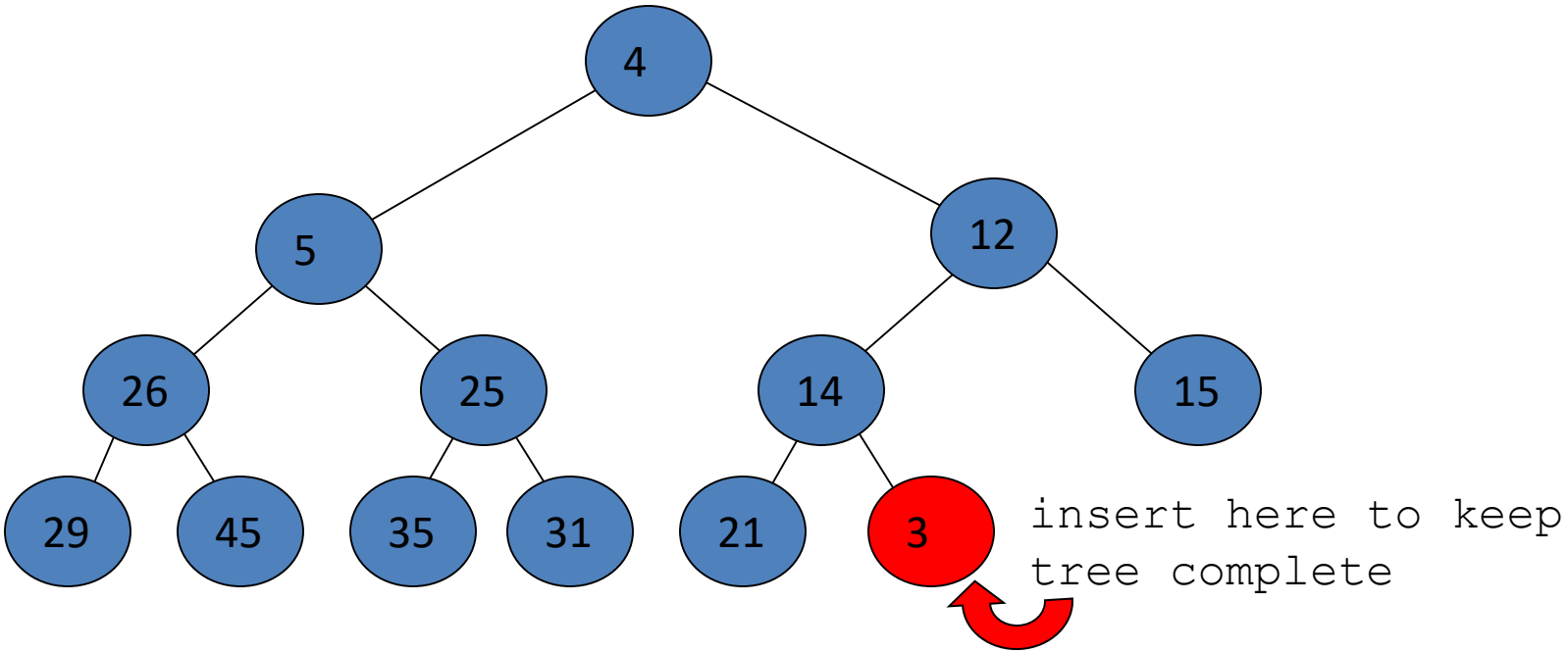
currentsize = 12

Implementatation

- Heap must be a *complete* tree

- all leaves are on the lowest two levels
- nodes are added on the lowest level, from left to right
- nodes are removed from the lowest level, from right to left

Inserting a Value

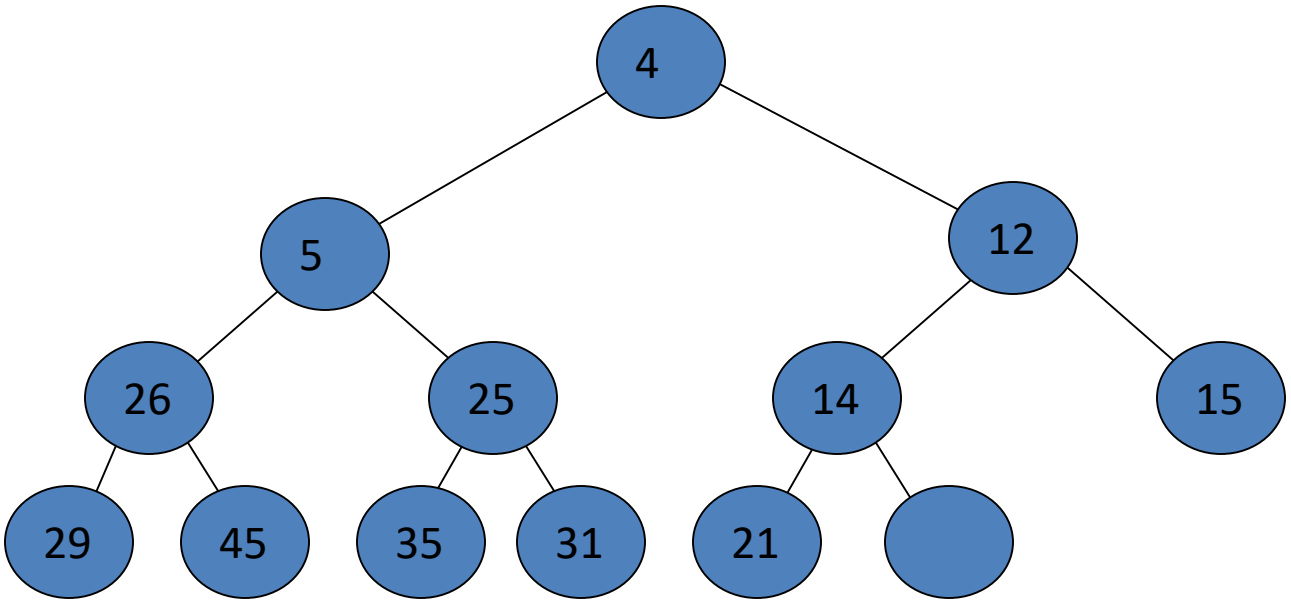


i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
array	__	4	5	12	26	25	14	15	29	45	35	31	21	3	__	__

currentsize = 13

Insert 3

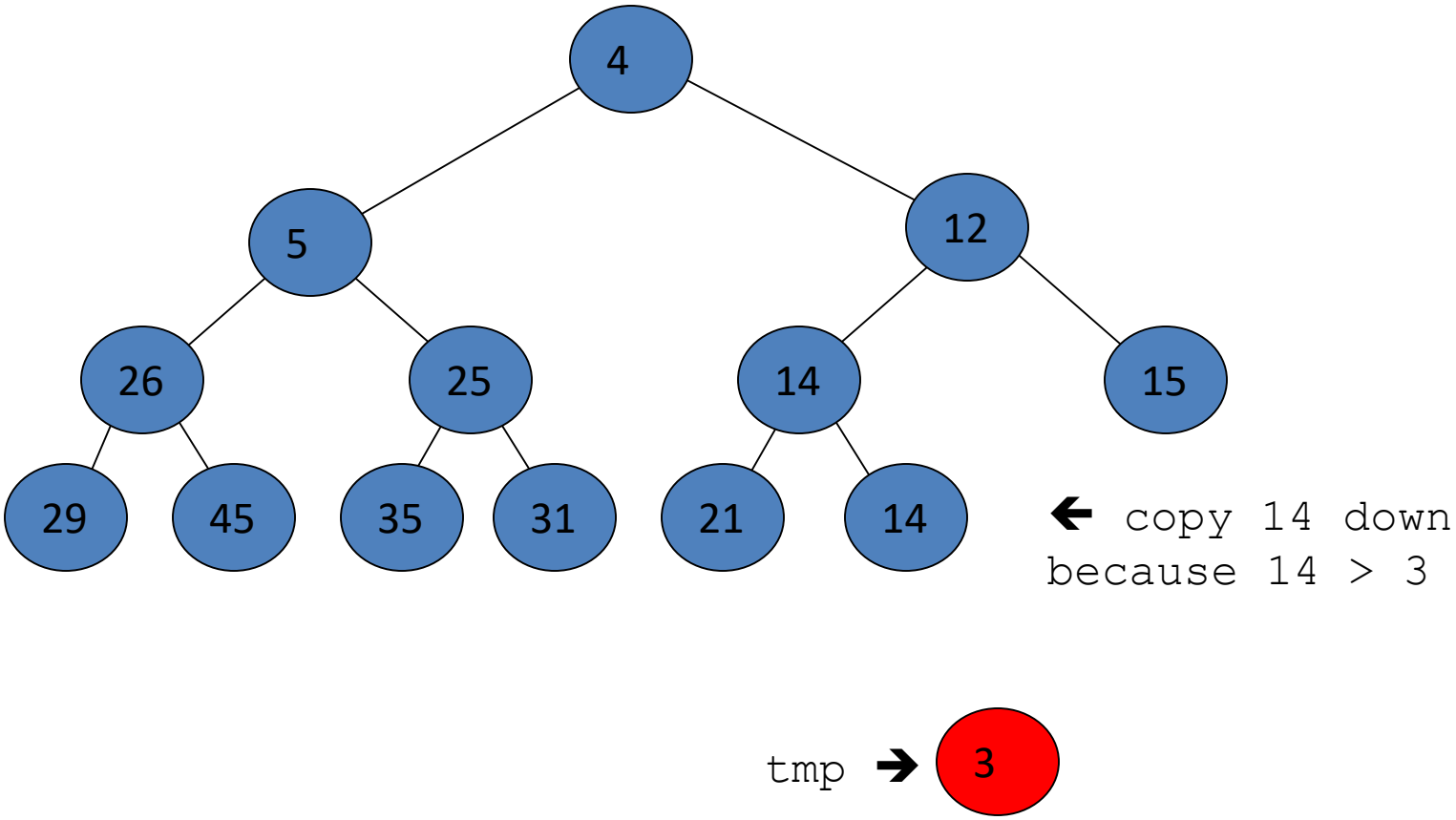
Inserting a Value



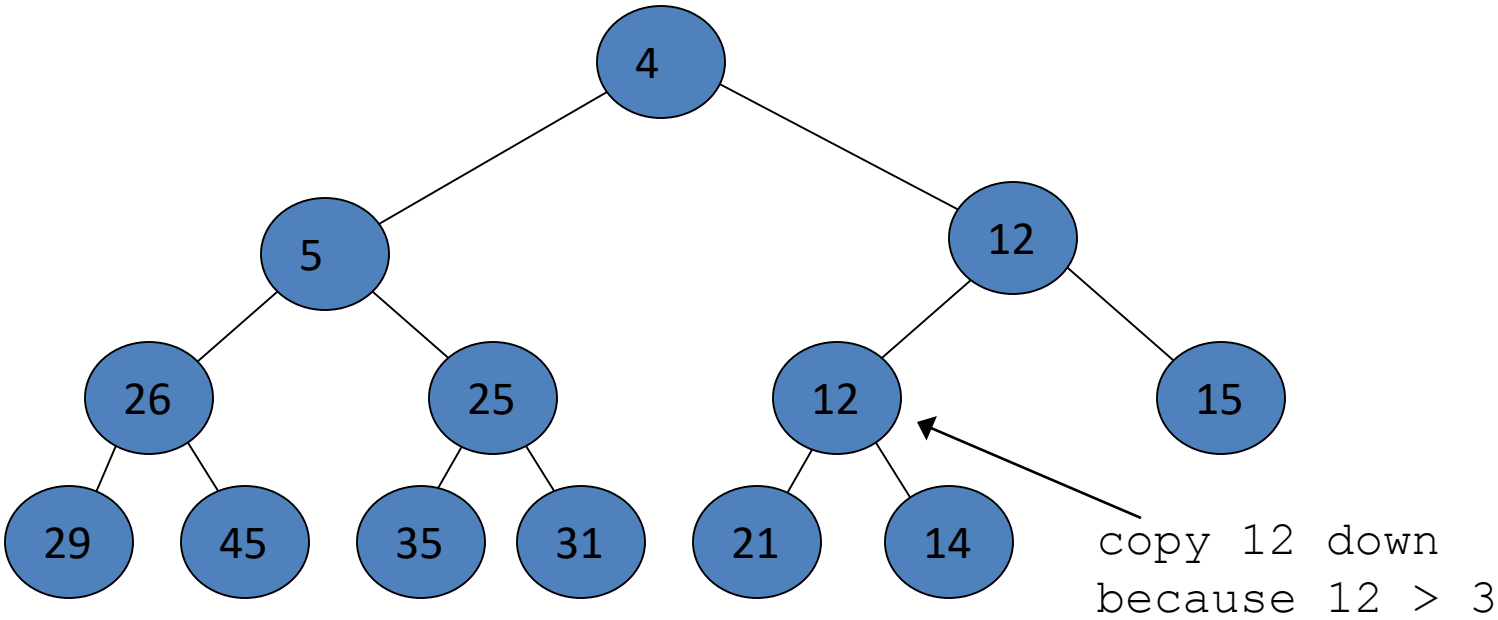

```
save new value in a
temporary location: tmp ➔
```

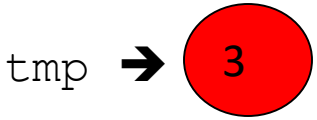
Insert 3

Inserting a Value



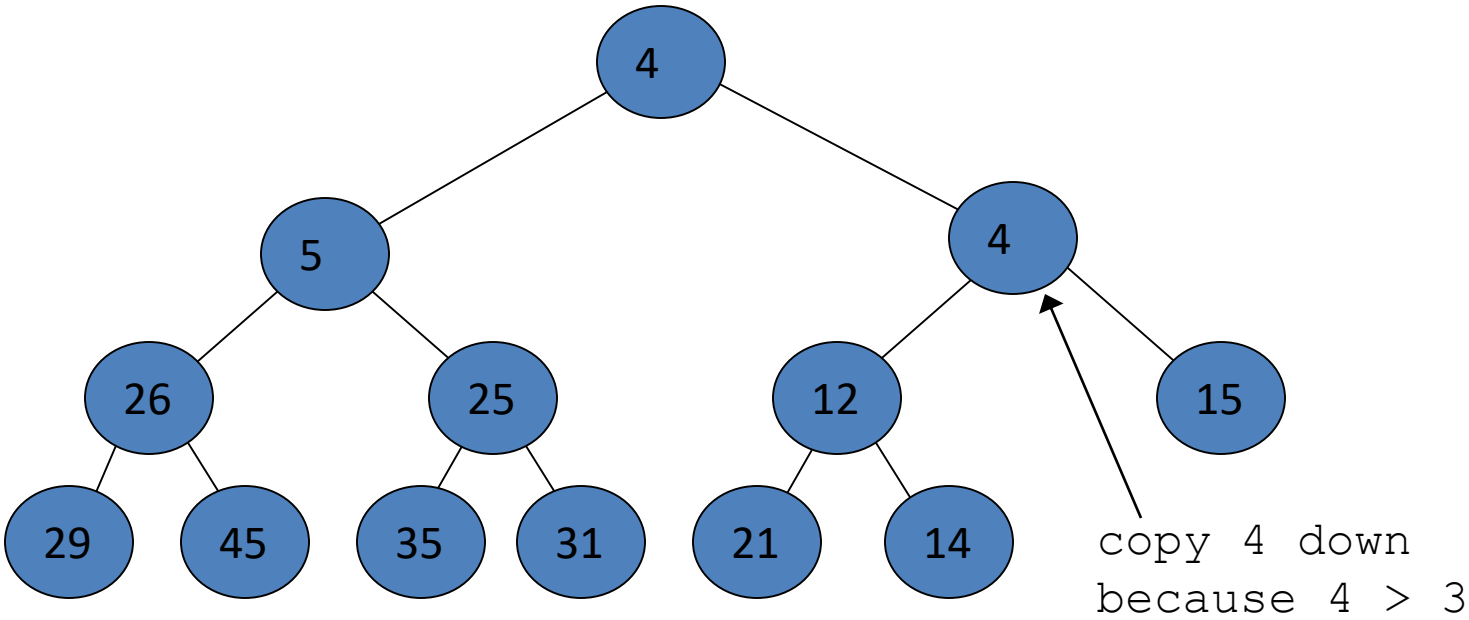
Inserting a Value

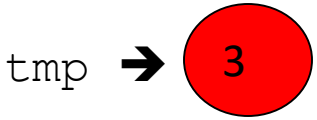




Insert 3

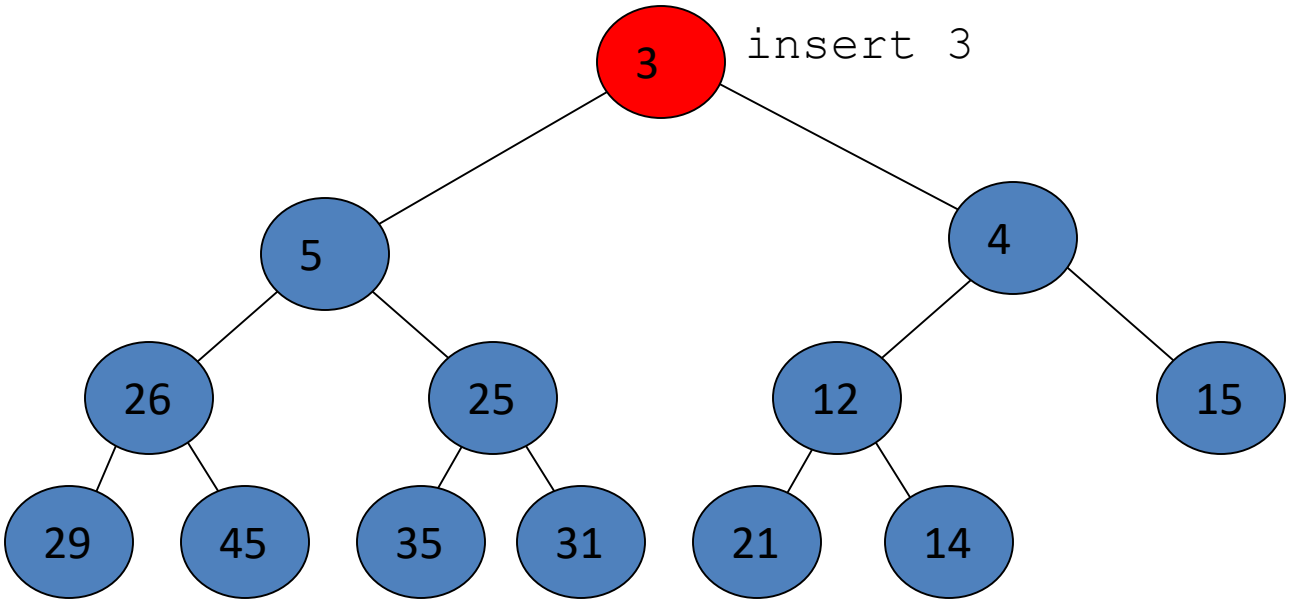
Inserting a Value





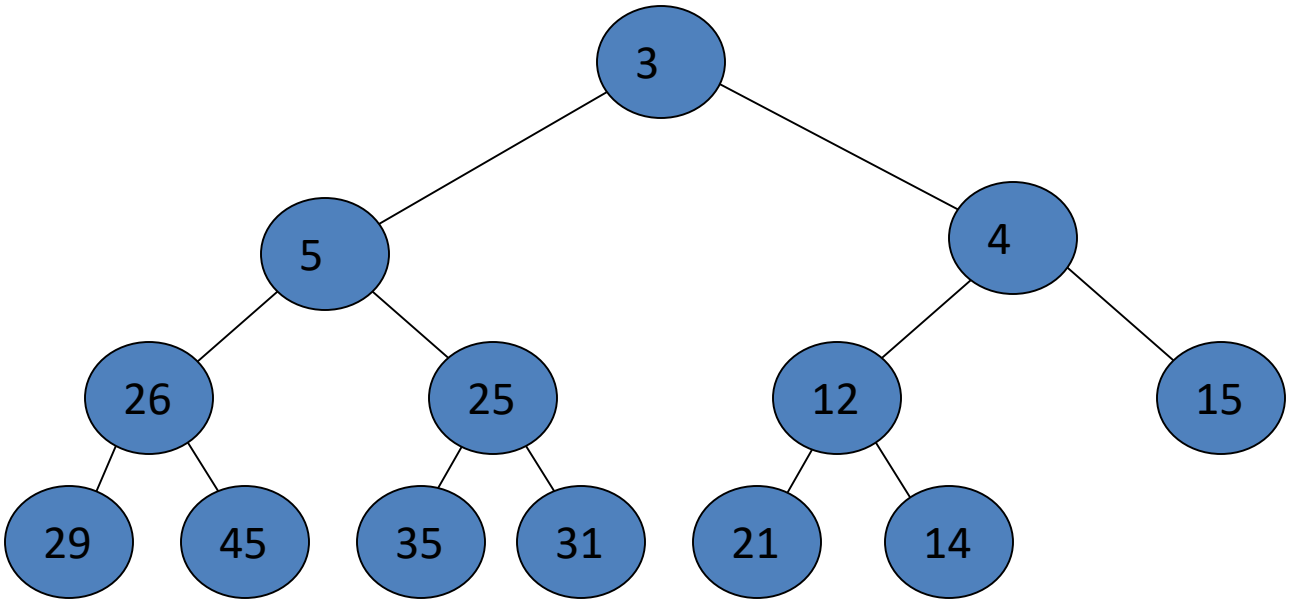
Insert 3

Inserting a Value

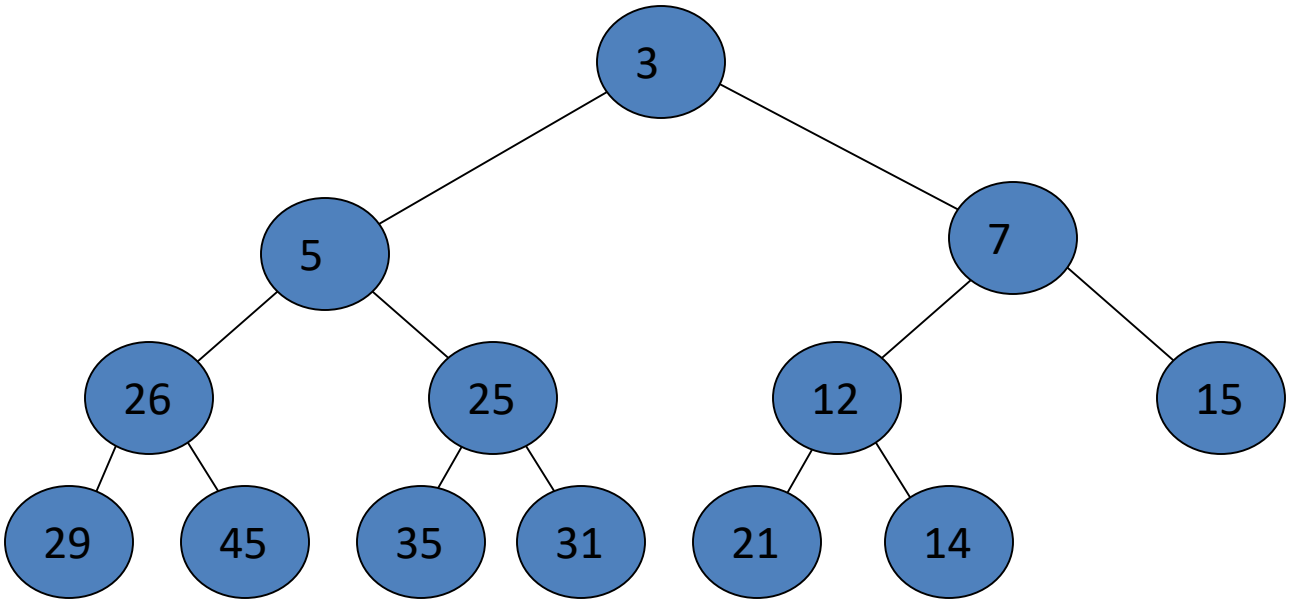


Insert 3

Heap After Insert

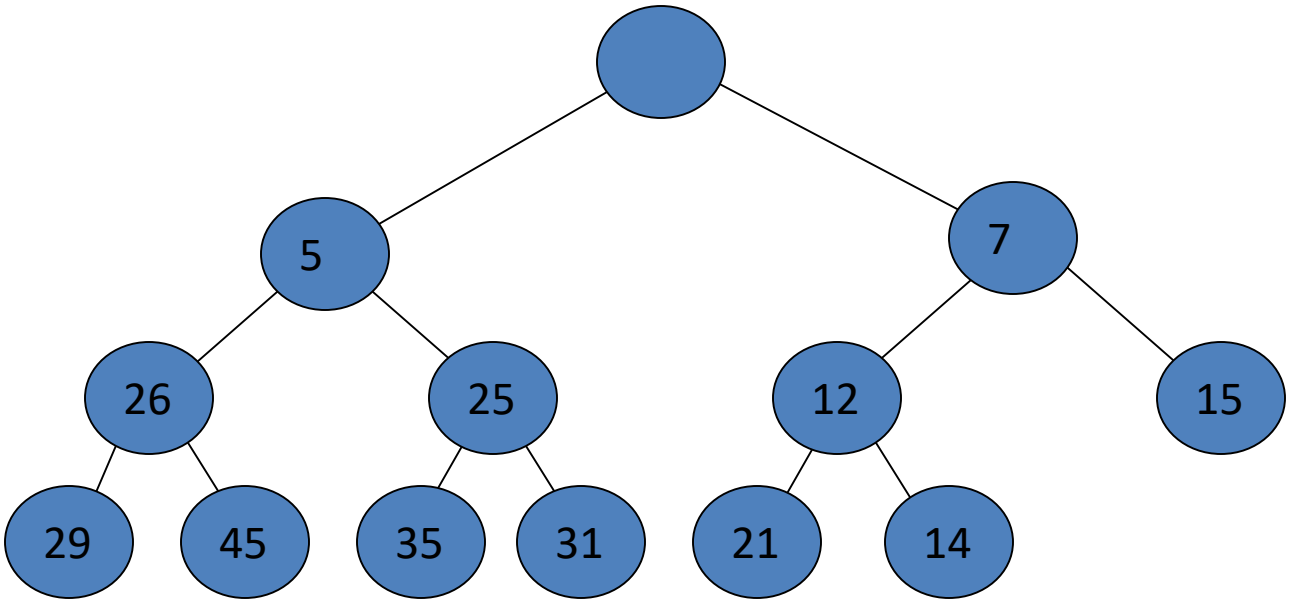


Deleting a Value (note new tree!)

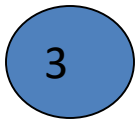


Delete 3

Deleting a Value



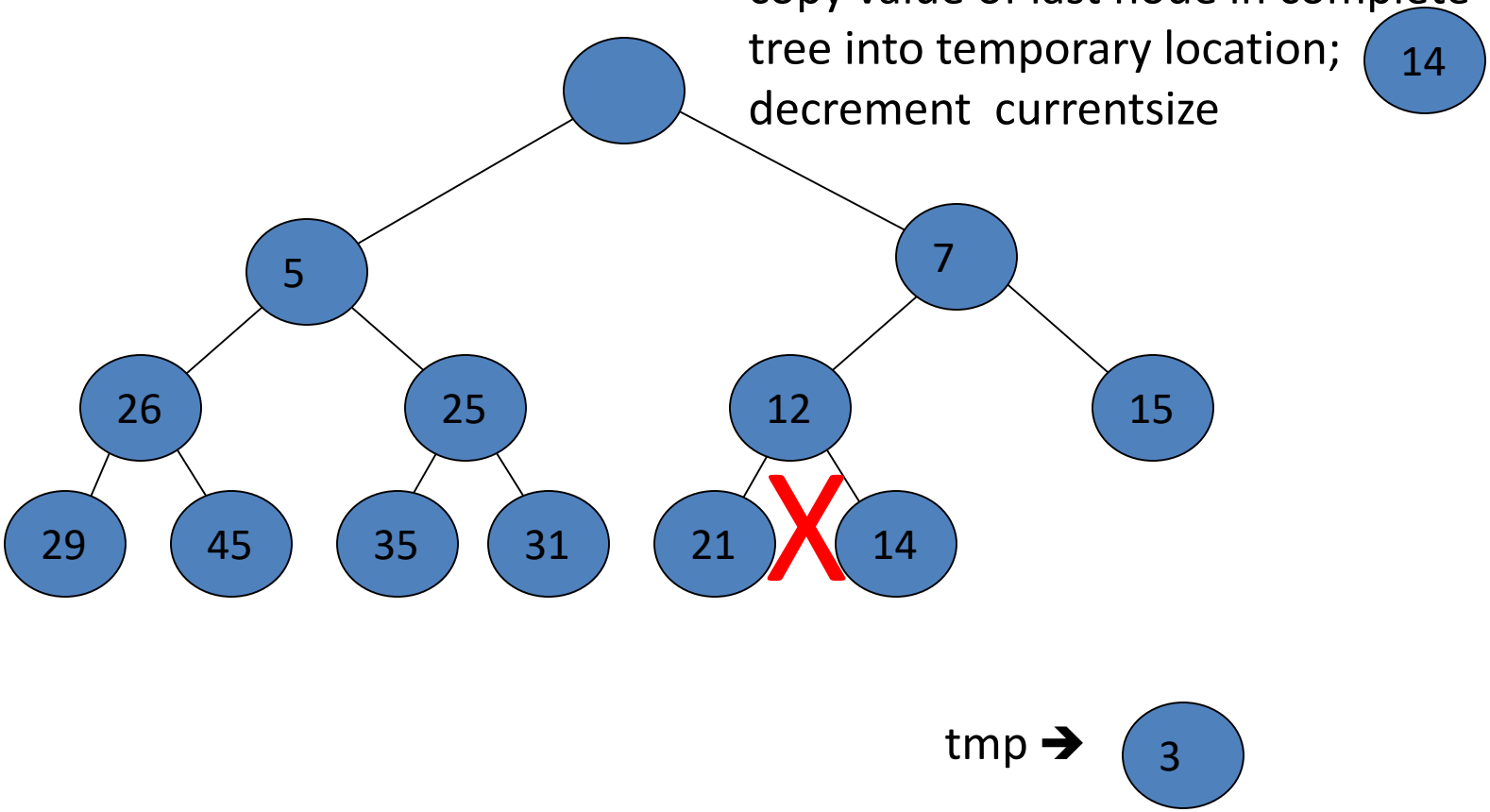
save root value … tmp ➔



Delete 3

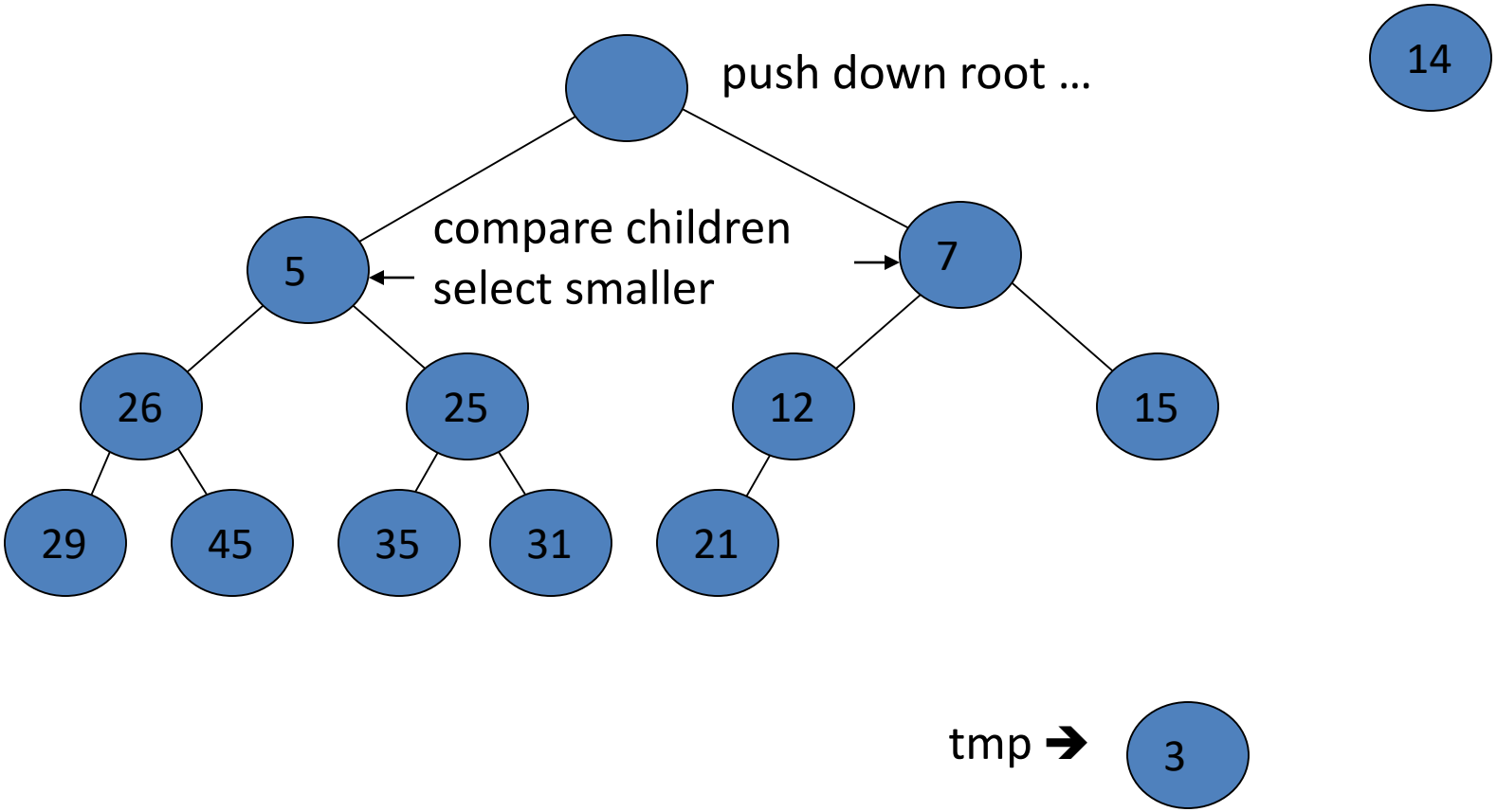
Deleting a Value

copy value of last node in complete tree into temporary location; decrement currentsize



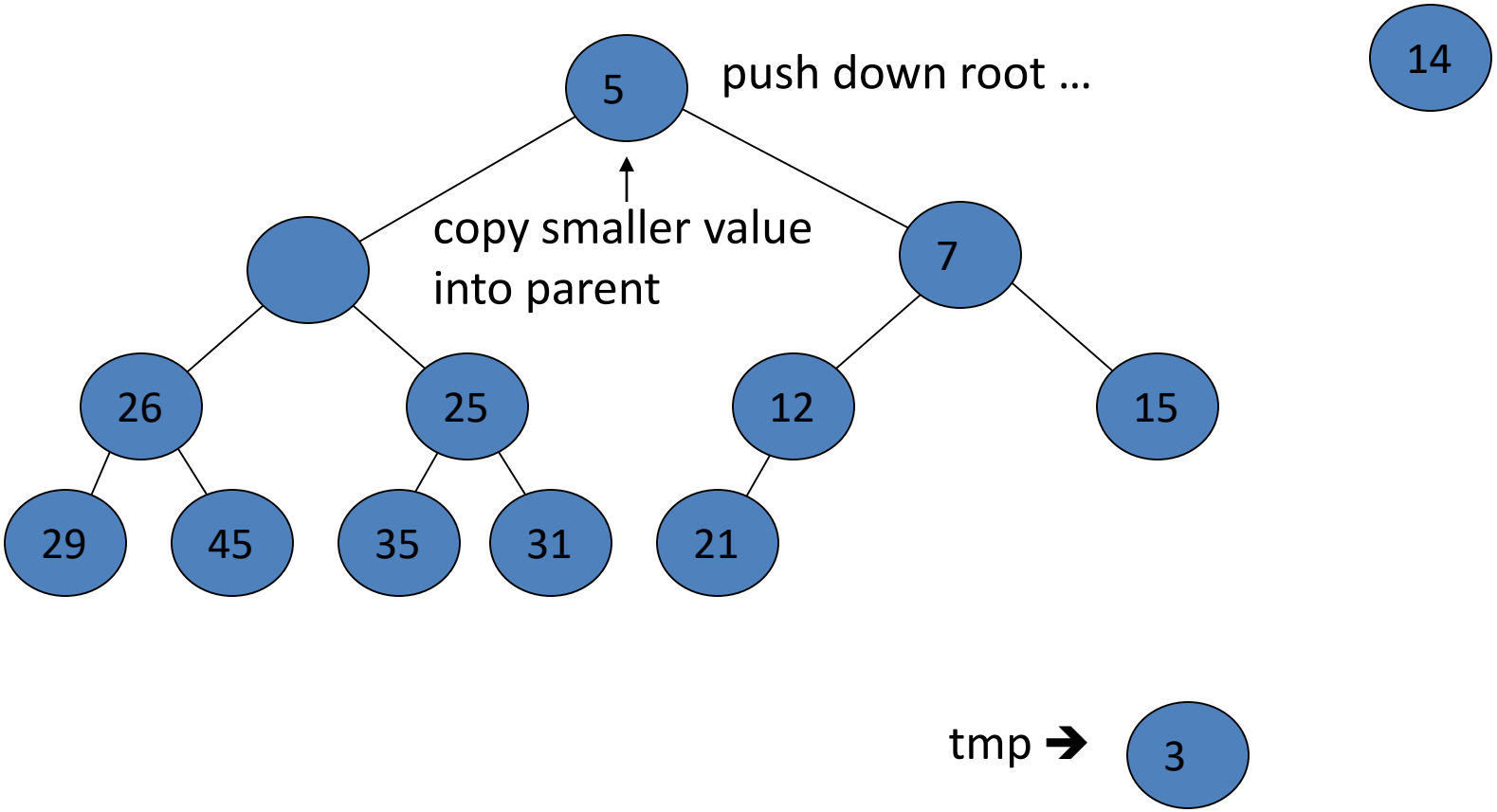
Delete 3

Deleting a Value



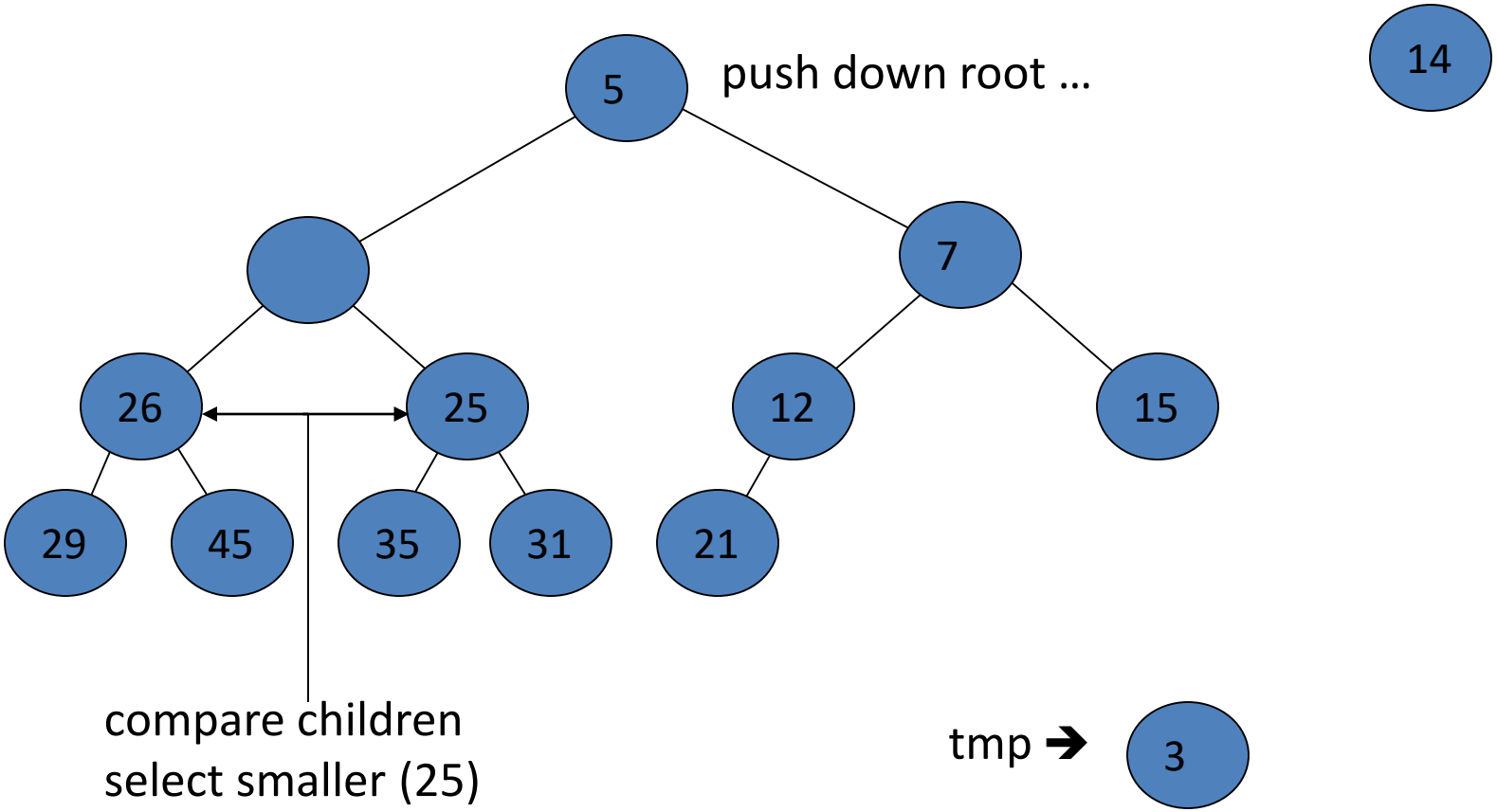
Delete 3

Deleting a Value



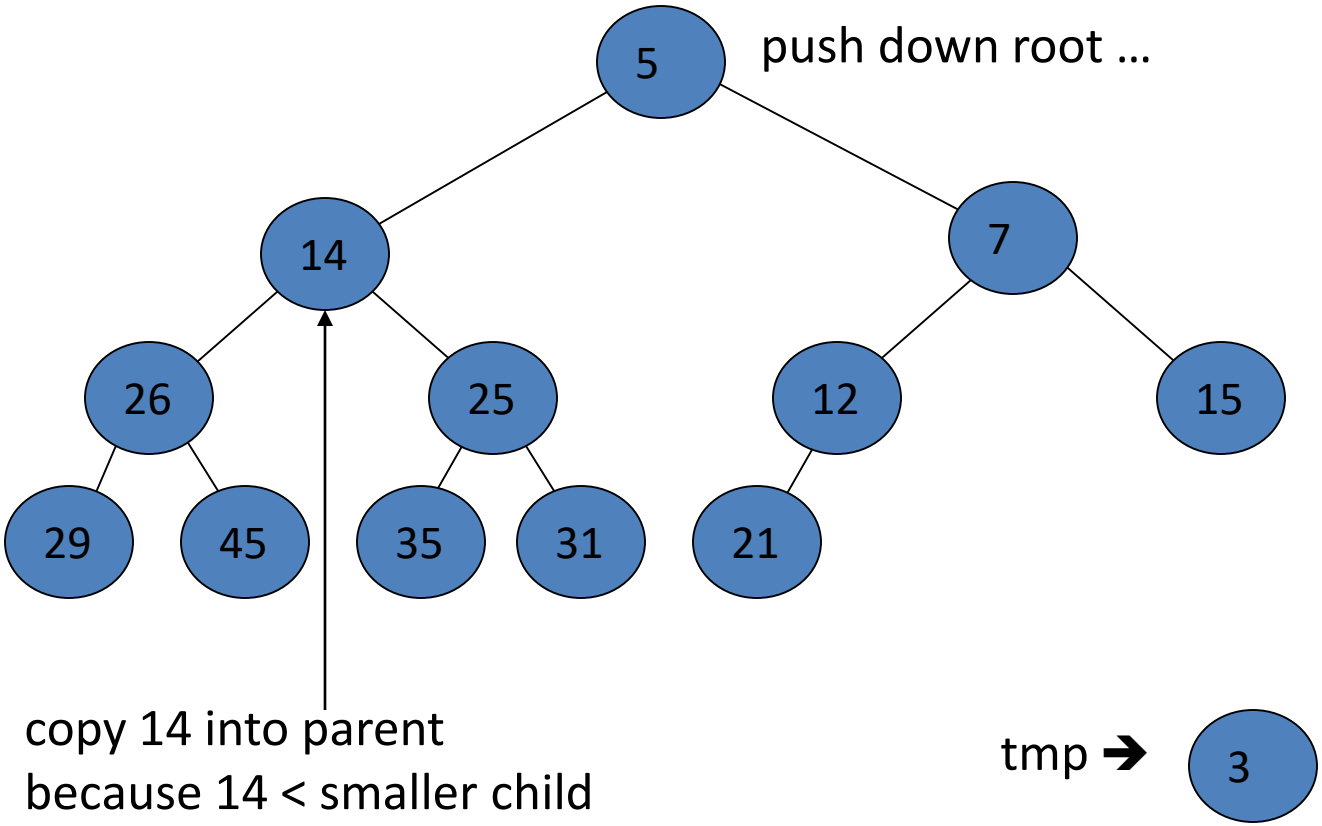
Delete 3

Deleting a Value



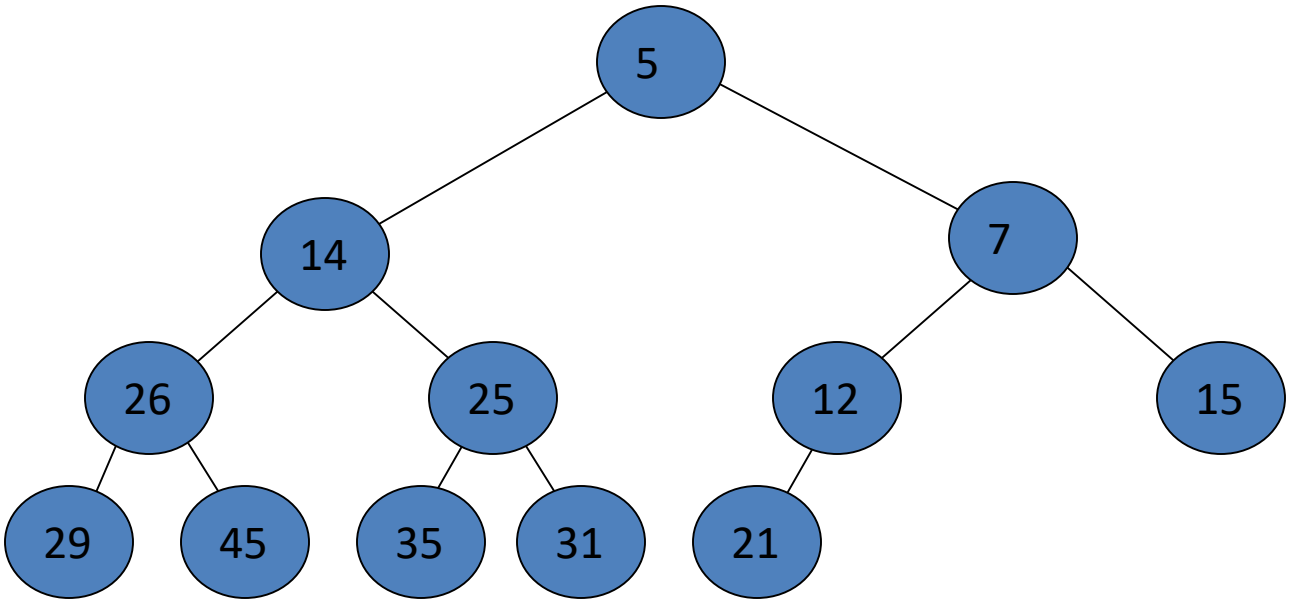
Delete 3

Deleting a Value

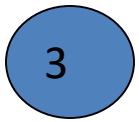


Delete 3

Deleting a Value



return



Delete 3